

Prognostisering av offhire for fartøy i offshoresegmentet

En sammenligning av SARIMA, eksponentiell glatting, XGBoost og LSTM

Forfatter(e): Julie Bjørheim

Totalt antall sider inkludert forsiden: [fylles inn ved slutføring]

Molde, innleveringsdato: [fylles inn ved slutføring]

Obligatorisk egenerklæring/gruppeerklæring

Denne delen følger malen fra Høgskolen i Molde og fylles ut i endelig Word-versjon før levering.

Personvern

- **Har oppgaven vært vurdert av NSD?** [ja/nei]
- **NSD referansenummer:** [fylles inn ved behov]
- **Har oppgaven vært til behandling hos REK?** [ja/nei]
- **REK referansenummer:** [fylles inn ved behov]

Publiseringsavtale

- **Studiepoeng:** [fylles inn ved slutføring]
 - **Veileder:** [fylles inn ved slutføring]
 - **Elektronisk publisering:** [ja/nei]
 - **Båndlagt (konfidensiell):** [ja/nei]
 - **Publisering etter båndleggingsperiode:** [ja/nei, hvis relevant]
 - **Dato:** [fylles inn ved slutføring]
 - **Antall ord:** [fylles inn hvis påkrevd]
 - **Forfattererklæring:** [fylles inn hvis påkrevd]
-

Sammendrag

Denne oppgaven undersøker hvordan valg av prognosemodell påvirker prediksjonsnøyaktigheten for offhire-hendelser for fartøy innenfor samme offshoresegment. Offhire representerer perioder med operasjonell nedetid eller manglende kontraktsinntekt, og utgjør et viktig beslutningsproblem i et marked preget av teknisk kompleksitet, kontraktsmessige forpliktelser og betydelig volatilitet. Studien er gjennomført som en kvantitativ, casebasert sammenligning av fire prognosemodeller: SARIMA, eksponentiell glatting, XGBoost og LSTM. Datagrunnlaget består av historiske, anonymiserte offhire-data for 16 fartøy. Modellene ble estimert og evaluert på samme historiske oppsett med et eksplisitt train/test-splitt og ekspanderende 1-steps prognoser gjennom testperioden. Prediksjonsnøyaktigheten ble vurdert ved hjelp av MAE, RMSE og SMAPE.

Resultatene viser at modellvalg har betydning for prediksjonsnøyaktigheten, men ikke på en måte som gir automatisk fordel til de mest komplekse modellene. ARIMA/SARIMA oppnådde lavest MAE og RMSE i den historiske testen, mens XGBoost og LSTM var konkurransedyktige uten å overgå den beste klassiske modellen. Eksponentiell glatting fungerte som en nyttig, men svakere benchmark. Fremtidsprognosene for 1, 3, 6 og 12 måneder fram viste samtidig at modellene ga ulike framtidsbilder, og at usikkerheten økte med lengre prognosehorisont. Studien konkluderer derfor med at klassiske tidsseriemodeller framstår som det mest forsvarlige førstevalget i denne casen, samtidig som prognoser bør brukes som beslutningsstøtte og tolkes med faglig skjønn.

Abstract

This thesis examines how the choice of forecasting model affects the predictive accuracy of offhire events for vessels operating within the same offshore segment. Offhire refers to periods of operational downtime or lost contract revenue and represents an important decision-support problem in a market characterized by technical complexity, contractual obligations, and substantial volatility. The study is designed as a quantitative, case-based comparison of four forecasting models: SARIMA, exponential smoothing, XGBoost, and LSTM. The empirical basis consists of historical, anonymized offhire data for 16 vessels. All models were estimated and evaluated under the same historical setup, using an explicit train/test split and expanding 1-step forecasts throughout the test period. Predictive performance was assessed using MAE, RMSE, and SMAPE.

The results show that model choice affects predictive accuracy, but not in a way that automatically favors the most complex models. ARIMA/SARIMA achieved the lowest MAE and RMSE in the historical test, while XGBoost and LSTM were competitive without outperforming the best classical model. Exponential smoothing served as a useful but weaker benchmark. The future forecasts for 1, 3, 6, and 12 months ahead also showed that the models produced different future paths and that uncertainty increased as the forecasting horizon became longer. The study therefore concludes that classical time-series models represent the most defensible first choice in this case, while forecasts should be used as decision support and interpreted with professional judgment.

Innhold

- 1.0 Innledning
 - 1.1 Problemstilling
 - 1.2 Avgrensninger
 - 1.3 Antakelser
- 2.0 Litteratur
- 3.0 Teori
 - 3.1 Prognostisering som beslutningsstøtte
 - 3.2 Tidsserier og sentrale komponenter
 - 3.3 Klassiske tidsseriemodeller
 - 3.4 SARIMA
 - 3.5 Eksponentiell glatting
 - 3.6 Maskinlæring og dyp læring i prognostisering
 - 3.7 XGBoost
 - 3.8 LSTM
 - 3.9 Modellvalg og sammenligningskriterier
- 4.0 Casebeskrivelse
- 5.0 Metode og data
 - 5.1 Metode
 - 5.1.1 Modellutvalg og evalueringsoppsett
 - 5.2 Data
 - 5.2.1 Datagrunnlag
 - 5.2.2 Deskriptiv analyse av datasettet
- 6.0 Modellering
 - 6.1 SARIMA
 - 6.2 Eksponentiell glatting
 - 6.3 XGBoost
 - 6.4 LSTM
 - 6.5 Oppsett for fremtidsprognoser
- 7.0 Resultater
 - 7.1 Resultater fra historisk modelltesting
 - 7.2 Resultater fra fremtidsprognoser
 - 7.2.1 Prognose 1 måned fram
 - 7.2.2 Prognose 3 måneder fram
 - 7.2.3 Prognose 6 måneder fram
 - 7.2.4 Prognose 12 måneder fram
- 8.0 Diskusjon
 - 8.1 Modellvalg og prediksjonsnøyaktighet
 - 8.2 Datastruktur, marked og hvorfor resultatene ble som de ble
 - 8.3 Fremtidsprognoser og praktisk tolkning
 - 8.4 Metodiske styrker og svakheter
 - 8.5 Betydning for bedriften og samlet vurdering

9.0 Konklusjon

10.0 Bibliografi

11.0 Vedlegg

11.1 Oversikt over figurer

11.2 Oversikt over tabeller

11.3 Kodevedlegg

11.3.1 SARIMA-kode

11.3.2 Eksponentiell glatting-kode

11.3.3 XGBoost-kode

11.3.4 LSTM-kode

1.0 Innledning

Fartøy som opererer i offshoresegmentet er en sentral del av infrastrukturen rundt petroleumsaktiviteten på norsk sokkel. De transporterer utstyr, drivstoff, forsyninger og andre innsatsfaktorer som er nødvendige for å opprettholde løpende aktivitet. Samtidig foregår operasjonene i et miljø preget av teknisk kompleksitet, kontraktsmessige forpliktelser og krevende operasjonelle rammevilkår. I denne konteksten kan perioder med nedetid, ofte omtalt som offhire, få betydelige konsekvenser både for rederiene og for oppdragsgiverne.

For rederier innebærer offhire ikke bare tapte charterinntekter, men også økt usikkerhet knyttet til planlegging, ressursutnyttelse, kontraktsoppfølging og konkurranseevne. For operatørene kan samme nedetid skape forstyrrelser i logistikk og drift. Behovet for mer presise prognoser blir særlig tydelig i et marked som samtidig er preget av betydelige svingninger i aktivitet, investeringsnivå og fartøyrater (Menon Economics, 2026). Bedre beslutningsstøtte for offhire er derfor ikke bare et teknisk analyseproblem, men også et praktisk spørsmål om operasjonell robusthet i et volatilt marked.

Tradisjonelt har prognostisering innen logistikk og operasjonsstyring i stor grad vært basert på klassiske statistiske modeller, særlig ARIMA-lignende modeller og eksponentiell glatting. Disse modellene er metodisk etablerte og egner seg godt når historiske tidsserier inneholder relativt stabile mønstre i nivå, trend og sesong (Gardner, 1985; Hyndman et al., 2002; Hyndman & Khandakar, 2008). Samtidig viser nyere forskning at maskinlæringsmodeller kan være konkurransedyktige eller bedre når datastrukturen er mer kompleks, heterogen eller ikke-lineær (Carbonneau et al., 2008; Schmid et al., 2025).

Innen maritim sektor har datadrevne metoder de siste årene særlig vært brukt til prediktivt vedlikehold, markedsprediksjoner og andre operasjonelle beslutningsproblemer. Forskning som direkte sammenligner tradisjonelle tidsseriemodeller og KI-baserte modeller for prognostisering av offhire på fartøynivå er derimot mer begrenset. Dette gjør det faglig relevant å undersøke hvordan ulike modellfamilier presterer i akkurat denne konteksten (Chu et al., 2024; Kalafatelis et al., 2025; Kjeldsberg & Munim, 2024).

1.1 Problemstilling

Formålet med denne studien er å undersøke hvordan modellvalg påvirker prognostisering av operasjonell nedetid i offshorenæringen. På bakgrunn av behovet for mer presis beslutningsstøtte i et volatilt marked formuleres følgende problemstilling:

Hvordan påvirker valg av prognosemodell prediksjonsnøyaktigheten for offhire-hendelser for fartøy innenfor samme offshoresegment, når maskinlæringsmodeller sammenlignes med tradisjonelle tidsseriemodeller?

1.2 Avgrensninger

Studien avgrenses til 16 anonymiserte fartøy som opererer innenfor samme offshoresegment. Fartøy utenfor dette segmentet inngår ikke i analysen. Avgrensningen er valgt for å sikre størst mulig sammenlignbarhet i operasjonelle rammebetingelser og kontraktsforhold, selv om fartøyene ikke nødvendigvis tilhører én og samme fartøytype.

Analysen er videre avgrenset til prediksjon av offhire-hendelser, definert som perioder der fartøy midlertidig ikke opererer i henhold til kontrakt eller ikke genererer forventet inntekt. Det skilles ikke videre mellom ulike årsakskategorier innen offhire, fordi formålet er å evaluere modellenes prediktive ytelse og ikke å analysere årsakssammenhenger.

Studien bygger på historiske operasjonelle data innen en definert tidsperiode. Eksterne forhold som energipriser, geopolitisk risiko og bredere markedsendringer modelleres ikke eksplisitt, men inngår bare i den grad de er indirekte reflektert i observasjonene.

Fokus ligger på sammenligning av modelltyper med hensyn til prediksjonsnøyaktighet. Implementeringskostnader, organisatoriske endringer og teknologisk integrasjon inngår ikke i analysen. Modellutvalget er avgrenset til fire modeller: to tradisjonelle metoder, SARIMA og eksponentiell glatting, og to KI-baserte modeller, XGBoost og LSTM.

1.3 Antakelser

Definisjon av offhire

Det antas at definisjonen av offhire er konsistent gjennom hele datamaterialet. Dette er nødvendig for at variasjon i datasettet skal kunne tolkes som uttrykk for reelle operasjonelle forhold og ikke som følge av endrede registreringsrutiner. Eventuelle strukturelle brudd i rapporteringspraksis fanges derfor ikke eksplisitt opp i analysen.

Historiske mønstre inneholder prediktiv informasjon

Det antas at historiske operasjonelle data inneholder mønstre som kan brukes til å predikere framtidige offhire-hendelser. Denne antakelsen ligger til grunn for både tidsseriemodeller og maskinlæringsmodeller. Analysen vurderer derfor modellenes evne til å utnytte eksisterende historisk struktur, men ikke deres evne til å forutsi strukturelle brudd utenfor datagrunnlaget.

Uavhengighet mellom fartøy

Det antas at observasjoner kan behandles som tilnærmet uavhengige mellom fartøy. Denne antakelsen forenkler modelleringen og gjør det mulig å sammenligne prediktiv ytelse uten å eksplisitt modellere flåteinteraksjoner. Eventuelle systematiske sammenhenger mellom fartøy, for eksempel felles teknisk design eller kontraktsstruktur, inngår dermed ikke eksplisitt i modellene.

Markedsusikkerhet reflekteres i historiske data

Det antas at markedsmessig usikkerhet, herunder svingninger i aktivitet og rater, indirekte er reflektert i de historiske dataene. Studien modellerer ikke slike forhold eksplisitt, men forutsetter at deler av effekten er synlig i observerte offhire-mønstre. Modellene evalueres derfor under historisk observerte markedsforhold, men ikke mot hypotetiske ekstreme scenarier.

Prediktiv fremfor kausal analyse

Det antas at formålet med analysen er prediksjon og ikke kausal forklaring. Modellenes prestasjon vurderes derfor ut fra prediktiv nøyaktighet og ikke ut fra om variablene representerer direkte årsakssammenhenger. Konsekvensen er at studien kan sammenligne modeller på likt grunnlag, men ikke trekke kausale konklusjoner om hvilke forhold som skaper offhire.

2.0 Litteratur

Litteraturen om prognostisering i logistikk og forsyningskjeder viser et tydelig skifte fra et nesten ensidig fokus på klassiske statistiske modeller til et bredere metoderepertoar der maskinlæring og dyp læring inngår som reelle alternativer. Douaioui et al. (2024) viser i en nyere oversikt at veksten i KI-baserte prognosestudier har vært særlig

sterk de siste årene, og at forskningen i økende grad retter seg mot datasett med ikke-lineariteter, strukturelle brudd og høy kompleksitet. Dette betyr likevel ikke at klassiske modeller har mistet sin relevans. Litteraturen peker snarere mot at modellvalg må forstås som et spørsmål om problemstruktur og datagenererende prosesser, ikke som et teknologisk kappløp der den mest avanserte modellen automatisk er best.

Denne nyanseringen blir tydelig i forskning på prognoser under ustabile omgivelser. Fildes et al. (2022) viser at forecasting etter COVID-19 i mindre grad kan bygge på en enkel videreføring av stabile historiske mønstre, og i større grad må håndtere brudd, skift og episodiske sjokk. M5-konkurransen har gitt et viktig empirisk grunnlag for samme diskusjon. Makridakis et al. (2022) viser at store benchmark-studier kan avdekke betydelige forskjeller i prediksjonsnøyaktighet når datastrukturen er rik og krevende. Samtidig advarer Kolassa (2022) mot å lese slike konkurranser som et generelt bevis på at mer komplekse modeller alltid gir høyest praktisk verdi. Forskningen peker dermed mot en vurdering der prediktiv nøyaktighet, robusthet, tolkbarhet og praktisk anvendbarhet må ses i sammenheng.

Direkte sammenligninger mellom statistiske modeller og maskinlæringsmodeller understøtter samme poeng. Schmid et al. (2025) viser i en simuleringsstudie for data-drevet logistikk at maskinlæringsmetoder særlig kommer til sin rett når datastrukturen er preget av ikke-linearitet, heterogenitet og forstyrrelser, mens tradisjonelle tidsseriemodeller fortsatt kan være fullt konkurransedyktige i mer regelmessige settinger. Forskningsbildet gir dermed ikke støtte til en enkel antakelse om at modellkompleksitet i seg selv er et kvalitetsmål. Det sentrale blir i stedet å sammenligne modellfamilier under samme evalueringsoppsett og på samme datasett.

Innen maritim forskning er anvendelsen av slike modeller økende, men tematikken er fortsatt relativt smal. Kalafatelis et al. (2025) viser at KI i maritim sektor i stor grad er brukt innen prediktivt vedlikehold, med fokus på komponentfeil, tilstandsmonitorering og teknisk tilgjengelighet. Chu et al. (2024) viser at **XGBoost** kan forbedre prediksjoner av vessel turnaround time i havnesammenheng, mens Kjeldsberg og Munim (2024) demonstrerer at AutoML og maskinlæringsmodeller kan brukes til å predikere PSV-fraktrater i et marked preget av flere samtidige og ikke-lineære drivere. Felles for disse studiene er at de dokumenterer økende bruk av datadrevne modeller i maritime beslutningsproblemer, men de retter seg hovedsakelig mot teknisk vedlikehold, havneoperasjoner eller markedsrater.

Det er derfor fortsatt et tydelig forskningsgap knyttet til prognostisering av operasjonell nedetid og offhire på fartøynivå i offshoresegmentet. Den foreliggende studien er motivert av dette gapet. I stedet for å teste én enkelt modell undersøker oppgaven hvordan to klassiske tidsseriemodeller, **SARIMA** og **eksponentiell glatting**, og to KI-baserte modeller, **XGBoost** og **LSTM**, presterer når de sammenlignes på samme datastruktur og med samme historiske evalueringslogikk. Historisk prediksjonsnøyaktighet brukes dermed som hovedgrunnlag for modellvurderingen, mens fremtidsprognosene tolkes i lys av denne historiske testen.

3.0 Teori

3.1 Prognostisering som beslutningsstøtte

Prognostisering er sentralt i logistikk og operasjonsstyring fordi beslutninger om ressursbruk, vedlikehold, kontrakter og kapasitet må tas før framtidige utfall er kjent. En prognose er derfor ikke bare et estimat av en framtidig verdi, men et beslutningsverktøy som reduserer usikkerhet under operasjonelle begrensninger. I denne studien er dette viktig fordi formålet er prediksjon av offhire-hendelser og ikke kausal forklaring av hvorfor de oppstår. Modellenes verdi vurderes dermed ut fra hvor godt de kan omsette historiske mønstre til praktisk beslutningsstøtte for framtidige perioder (Carbonneau et al., 2008; Fildes et al., 2022).

3.2 Tidsserier og sentrale komponenter

En tidsserie er en sekvens av observasjoner y_t ordnet i tid, der rekkefølgen er analytisk meningsbærende. Observasjoner som ligger nær hverandre i tid vil ofte være statistisk avhengige, og denne avhengigheten er selve grunnlaget for prognostisering. I klassisk tidsserieanalyse dekomponeres serien gjerne i fire hovedkomponenter: nivå, trend, sesong og et irregulært restledd. Nivå beskriver seriens typiske størrelse, trend beskriver en mer langsiktig utvikling, sesong beskriver gjentakende mønstre med fast periode, og restleddet representerer variasjon som ikke fanges av den systematiske strukturen.

For offhire-data er denne dekomponeringen relevant fordi materialet kan inneholde både perioder med stabilt lavt nivå, episodiske topper og kalendermessige mønstre knyttet til drift og kontraktsforhold. Samtidig er det viktig å skille mellom modellfamiliene i hvordan de bruker slik struktur. Klassiske tidsseriemodeller forsøker å spesifisere den direkte gjennom differensiering, glatting og autokorrelasjon, mens maskinlæringsmodeller og sekvensmodeller i større grad forsøker å lære mønstrene indirekte fra data. Like fullt er begrepene nivå, trend, sesong og støy et nyttig felles språk for å forstå prognoseproblemet på tvers av modelltyper (Gardner, 1985; Hyndman & Khandakar, 2008).

3.3 Klassiske tidsseriemodeller

Klassiske tidsseriemodeller bygger på at framtidige observasjoner kan estimeres ved å modellere den interne dynamikken i seriens egen historikk. I denne oppgaven representeres denne tradisjonen av **SARIMA** og **eksponentiell glatting**. Felles for dem er at de i hovedsak er univariate og lar prognosen bestemmes av tidligere observasjoner, eventuelle differensieringer og et begrenset sett av parametere eller tilstandskomponenter. Dette gjør dem relativt transparente sammenlignet med mer komplekse maskinlæringsmodeller.

En sentral styrke ved klassiske modeller er at antakelsene kan formuleres eksplisitt. Dersom tidsserien etter transformasjoner og differensieringer kan behandles som tilnærmet stabil, kan modeller med få parametere gi presise og tolkbare prognoser. Samtidig er denne styrken også en begrensning: når dataserien er sterkt uregelmessig, svært nulltung eller påvirkes av flere samtidige og ikke-lineære forhold, blir det vanskeligere å beskrive hele prognoseproblemet gjennom én eksplisitt tidsseriedynamikk. Valget av klassiske modeller i denne studien er derfor ikke begrunnet i at de nødvendigvis er enklest, men i at de representerer et parsimonisk og faglig veletablert sammenligningsgrunnlag (Gardner, 1985; Hyndman et al., 2002; Hyndman & Khandakar, 2008).

3.4 SARIMA

SARIMA, seasonal autoregressive integrated moving average, utvider ARIMA-rammeverket til serier med sesongmønstre. En $\text{SARIMA}(p, d, q)(P, D, Q)_S$ -modell kombinerer autoregressive ledd (p, P), differensiering (d, D) og glidende gjennomsnittsledd (q, Q) på både ordinært og sesongmessig nivå, der S er sesonglengden.

Standard matematisk form

$$\Phi(B^{12})\phi(B)(1-B)^d(1-B^{12})^D y_t = \Theta(B^{12})\theta(B)\varepsilon_t \quad (3.1)$$

Forklaring av symbolene

Her er y_t observert offhire-prosent i måned t , og B er backshift-operatoren, slik at $By_t = y_{t-1}$. Videre er $\phi(B)$ og $\theta(B)$ henholdsvis ikke-sesong autoregressivt og glidende gjennomsnittspolynom av orden p og q , mens $\Phi(B^{12})$ og $\Theta(B^{12})$ er sesongpolynomer av orden P og Q . Parameterne d og D angir ordinær og sesongmessig differensiering, ε_t er et tilfeldig feilledd, og sesonglengden er satt til 12 fordi dataseriene er månedlige.

Teoretisk er **SARIMA** mest relevant når historikken inneholder en tidsstruktur som kan beskrives gjennom autokorrelasjon og gjentakende sesongmønstre. Modellen er derfor sterk når nivå, trend og sesong kan identifiseres relativt klart, og når en viktig del av prognoseproblemet ligger i seriens egen dynamikk. Samtidig er modellen sårbar dersom serien er kort, svært nulltung eller dominert av uregelmessige sprang, fordi antakelsen om en tilnærmet stabil tidsserieprosess da blir vanskeligere å opprettholde (Hyndman & Khandakar, 2008).

3.5 Eksponentiell glatting

Eksponentiell glatting bygger på en annen modelllogikk enn **SARIMA**, men er like fullt en sentral klassisk prognosefamilie. Grunntanken er at nyere observasjoner skal få høyere vekt enn eldre observasjoner, der vektene avtar eksponentielt bakover i tid. I den moderne **ETS**-forståelsen beskrives modellen gjennom uobserverte tilstandskomponenter for nivå, trend og eventuelt sesong, som oppdateres rekursivt for hver ny observasjon (Gardner, 1985; Hyndman et al., 2002).

Standard matematisk form

$\ell_t = \alpha(y_t - s_{t-12}) + (1 - \alpha)(\ell_{t-1} + b_{t-1})$	(3.2)
$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}$	(3.3)
$s_t = \gamma(y_t - \ell_t) + (1 - \gamma)s_{t-12}$	(3.4)
$\widehat{y}_{t+1 t} = \ell_t + b_t + s_{t+1-12}$	(3.5)

Forklaring av symbolene

Her er y_t observert offhire-prosent i måned t , ℓ_t nivåkomponenten, b_t trendkomponenten og s_t sesongkomponenten. Parametrene α , β og γ er glattingsparametere mellom 0 og 1 som styrer hvor raskt nivå, trend og sesong reagerer på ny informasjon. Prognosen $\widehat{y}_{t+1|t}$ uttrykker forventet verdi neste måned gitt informasjon tilgjengelig ved tid t .

For additive modeller blir prognosen et uttrykk for den løpende estimerte tilstanden i serien, snarere enn for en eksplisitt autokorrelasjonsstruktur. I praksis gjør dette **ETS**-modeller godt egnet som transparente og robuste benchmarker, særlig når formålet er å sammenligne enkle nivå-, trend- og sesongrepresentasjoner mot mer komplekse modeller.

I denne studien er eksponentiell glatting teoretisk relevant fordi modellfamilien representerer en parsimonisk mellomposisjon: den er mindre strukturert enn **SARIMA**, men langt mer transparent enn **XGBoost** og **LSTM**. Dersom nyere offhire-observasjoner faktisk bærer mest relevant informasjon om den nære framtiden, kan modellen gi konkurransedyktige prognoser med få parametere. Dersom datasettet derimot domineres av episodiske sprang og høy heterogenitet mellom fartøy, vil modellfamilien lettere bli for konservativ og underreagere på ekstreme utslag (Gardner, 1985; Hyndman et al., 2002).

3.6 Maskinlæring og dyp læring i prognostisering

Maskinlæring og dyp læring representerer en mer datadrevet tilnærming til prognostisering enn klassiske tidsseriemodeller. I stedet for å anta en bestemt stokastisk struktur for serien, forsøker modellene å lære en funksjonell sammenheng mellom input x_t og output y_t direkte fra data. Dette gjør dem særlig relevante når

problemet preges av ikke-linearitet, interaksjoner mellom flere trekk eller heterogenitet som er vanskelig å beskrive med få eksplisitte parametere.

I prognosesammenheng er det nyttig å skille mellom feature-baserte modeller og sekvensbaserte modeller. Feature-baserte modeller, som **XGBoost**, krever at tidsproblemet omformes til et sett av eksplisitte inputvariabler, for eksempel laggede verdier, rullerende gjennomsnitt og kalenderindikatorer. Sekvensbaserte modeller, som LSTM, lar i større grad modellen lære tidsavhengigheter direkte fra ordnede sekvenser av observasjoner. Felles for disse modellene er at de tilbyr høy fleksibilitet, men også høyere krav til datamengde, datakvalitet og modelloppsett enn de klassiske tidsseriemodellene (Carbonneau et al., 2008; Douaioui et al., 2024).

3.7 XGBoost

XGBoost er en trebasert maskinlæringsmodell bygget på gradient boosting. Modellen konstruerer en additiv funksjon der prediksjonen skrives som summen av mange beslutningstrær.

Standard matematisk form

$$\widehat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}$$

(3.6)

$$\mathcal{L}(\phi) = \sum_{i=1}^n l(y_i, \widehat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

(3.7)

$$\Omega(f_k) = \gamma T_k + \frac{1}{2} \lambda \|w_k\|^2$$

(3.8)

Forklaring av symbolene

Her er y_i observert offhire-prosent for observasjon i , $\widehat{y}_i$ modellens prediksjon, og x_i feature-vektoren som beskriver observasjonen. Hvert f_k representerer et beslutningstre, K er antall trær, og $\mathcal{F}$ er rommet av mulige regresjonstrær. Tapsfunksjonen $l(y_i, \widehat{y}_i)$ måler prediksjonsfeil, mens regulariseringsleddet $\Omega(f_k)$ straffer unødvendig komplekse trær gjennom antall terminale noder T_k og bladvektor w_k , styrt av parametrene γ og λ .

Chen og Guestrin (2016) viser at **XGBoost** kombinerer høy prediksjonsstyrke med regularisering, effektiv trebygging og håndtering av sparsitet. Regulariseringsleddet i objektfunksjonen gjør at modellen ikke bare forsøker å minimere treningsfeil, men også straffer unødige komplekse trær.

I tidsserieprognoser er **XGBoost** teoretisk interessant fordi modellen ikke er sekvensiell i seg selv. Den må få all tidsinformasjon gjennom eksplisitt feature engineering, for eksempel laggede verdier, rullerende mål og kalenderrepresentasjoner. Dette betyr at modellens styrke ligger i evnen til å utnytte ikke-lineariteter og interaksjoner i et konstruert feature-rom, ikke i en innebygd tidsserieforståelse. I denne studien gjør dette **XGBoost** til en relevant kontrast til både **SARIMA** og **eksponentiell glatting**: dersom offhire best forstås som et ikke-lineært panelproblem med fartøyspesifikke effekter, bør modellen ha et teoretisk fortrinn. Dersom den relevante strukturen først og fremst ligger i hver seriers interne dynamikk, kan behovet for eksplisitt feature engineering bli en begrensning (Chen & Guestrin, 2016).

3.8 LSTM

LSTM, long short-term memory, er en rekurrent sekvensmodell utviklet for å lære avhengigheter over lengre tidshorisonter. Hochreiter og Schmidhuber (1995) utviklet modellen for å håndtere problemet med at vanlige rekurrente nettverk har vansker med å bevare eller propagere relevant informasjon over lange sekvenser. Kjernen i

LSTM er en minnecelle c_t og en skjult tilstand h_t , styrt av inngangs-, glemme- og utgangsporter.

Standard matematisk form

$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	(3.9)
---	-------

$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	(3.10)
---	--------

$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$	(3.11)
--	--------

$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	(3.12)
---	--------

$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	(3.13)
---	--------

$h_t = o_t \odot \tanh(c_t)$	(3.14)
------------------------------	--------

$\hat{y}_{t+1} = W_y h_t + b_y$	(3.15)
---------------------------------	--------

Forklaring av symbolene

Her er x_t inputvektoren ved tid t , f_t forget gate, i_t input gate, $\tilde{c}_t$ kandidat for ny celletilstand, c_t celletilstanden, o_t output gate og h_t den skjulte tilstanden. Symbolene $\sigma(\cdot)$ og $\tanh(\cdot)$ betegner aktiveringsfunksjoner, $\odot$ er elementvis multiplikasjon, og W samt b er vektor og biasledd som estimeres i treningen. Prognosen $\hat{y}_{t+1}$ uttrykker forventet offhire-prosent i neste måned gitt sekvensinformasjonen fram til tid t .

Portene regulerer hvilken informasjon som beholdes, oppdateres og eksponeres videre i sekvensen. Denne strukturen gjør modellen fundamentalt forskjellig fra feature-baserte maskinlæringsmodeller: tidsavhengigheten læres i selve nettverket og trenger ikke kodes fullt ut manuelt som laggede features.

I denne oppgaven er LSTM teoretisk relevant fordi modellen representerer den mest fleksible og sekvensorienterte måten å lære mønstre i offhire-data på. Dersom fartøyenes historikk inneholder lange eller sammensatte avhengigheter som ikke lett lar seg beskrive gjennom eksplisitte lag og lineære parametere, bør LSTM i prinsippet kunne fange dette. Samtidig kommer denne fleksibiliteten med klare kostnader i form av større datakrav, høyere treningssensitivitet og lavere tolkbarhet enn både klassiske modeller og XGBoost. Modellen er derfor faglig interessant nettopp fordi den utfordrer spørsmålet om hvor mye kompleksitet datasettet faktisk bærer (Hochreiter & Schmidhuber, 1995).

3.9 Modellvalg og sammenligningskriterier

Et sentralt spørsmål i prognoseteori er hvordan modeller skal sammenlignes på en rettferdig måte. Makridakis et al. (2022) viser gjennom M5-konkurransen at modellvalg har stor betydning for prognoseytelsen i komplekse datasett, men også at rangeringen av modeller avhenger av både datastruktur og evalueringskriterier. Kolassa (2022) understreker samtidig at modellkompleksitet ikke er et kvalitetsmål i seg selv. En metodisk forsvarlig sammenligning forutsetter derfor at modellene vurderes på likt informasjonsgrunnlag, med samme prognosehorisont og med evalueringsmål som faktisk belyser ulike sider av prognosekvalitet.

I prognosestudier brukes ofte flere feilmål samtidig fordi de fanger ulike egenskaper ved modellene. **MAE** uttrykker gjennomsnittlig absolutt avvik i original skala og er lett å tolke. **RMSE** kvadrerer avvikene og gir derfor større vekt til store feil. **SMAPE** brukes ofte som et skaleringsuavhengig prosentmål, men kan være mer krevende å tolke når serien inneholder mange null- eller nær-nullverdier. Samlet betyr dette at modellvurdering normalt må forstås som en avveining mellom prediktiv nøyaktighet, robusthet, tolkbarhet og datakrav, heller enn som en jakt på lavest mulig verdi i én enkelt metrikk (Kolassa, 2022; Schmid et al., 2025).

4.0 Casebeskrivelse

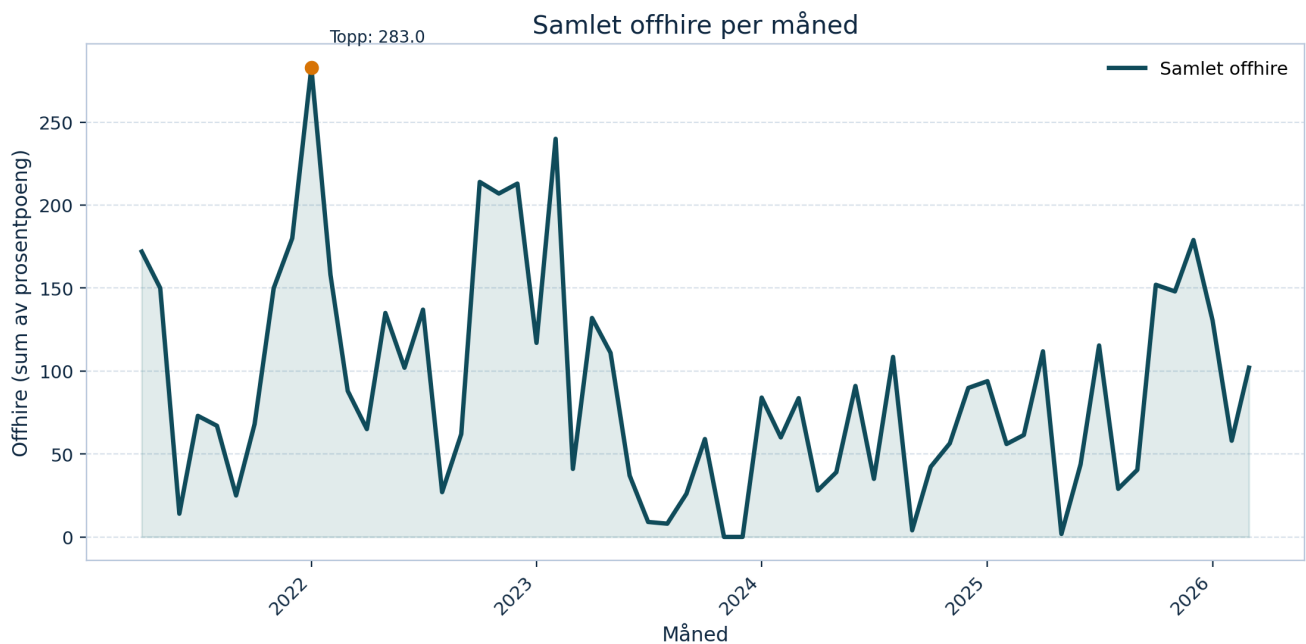
Denne studien tar utgangspunkt i Simon Møkster Shipping AS, et rederi som opererer innenfor offshoresegmentet. I denne konteksten er fartøyene en sentral del av verdikjeden fordi de understøtter aktivitetene på norsk sokkel gjennom løpende operasjoner, kontrakter og leveranser. For et rederi i dette segmentet er høy operasjonell tilgjengelighet avgjørende, siden fartøyenes verdi i stor grad er knyttet til evnen til å opprettholde kontrakter og levere stabile tjenester over tid.

Et sentralt problem i denne sammenhengen er offhire. I denne oppgaven defineres offhire som en periode hvor et fartøy midlertidig ikke kan operere i henhold til kontrakt eller ikke genererer forventet inntekt. Dette kan skyldes tekniske feil, vedlikehold, sertifikatforhold, operasjonelle avvik eller perioder uten kontrakt. For Simon Møkster Shipping AS innebærer slike perioder ikke bare direkte økonomiske konsekvenser, men også økt usikkerhet i planlegging, ressursutnyttelse, vedlikeholdsvurderinger og kontraktsoppfølging. Mer presise prognoser kan derfor ha praktisk verdi som beslutningsstøtte.

Studien er avgrenset til 16 fartøy innenfor samme segment i rederiet. Dette er gjort for å sikre at analysen bygger på observasjoner fra fartøy med relativt like operasjonelle rammebetingelser, selv om fartøyene ikke nødvendigvis er av samme type. Et viktig trekk ved caset er samtidig at offhire sjelden skyldes én enkelt faktor. Teknisk tilstand, driftsmønster, kontraktssituasjon og markedsforhold kan virke sammen, noe som gjør caset relevant for å sammenligne både tradisjonelle tidsseriemodeller og KI-baserte prognosemodeller.

Casebedriften opererer samtidig i et marked preget av betydelig volatilitet. Offshoreaktivitet påvirkes ikke bare av tekniske og operative forhold, men også av svingninger i energipriser, investeringsnivå, kontraktsaktivitet og globale markedsforhold. Dette gjør planlegging av fartøyutnyttelse og tilgjengelighet mer krevende, og øker den praktiske verdien av prognoser for offhire. Studien modellerer ikke slike eksterne forhold eksplisitt, men de utgjør en viktig del av konteksten som gjør prognoseproblemet beslutningsmessig relevant (Menon Economics, 2026).

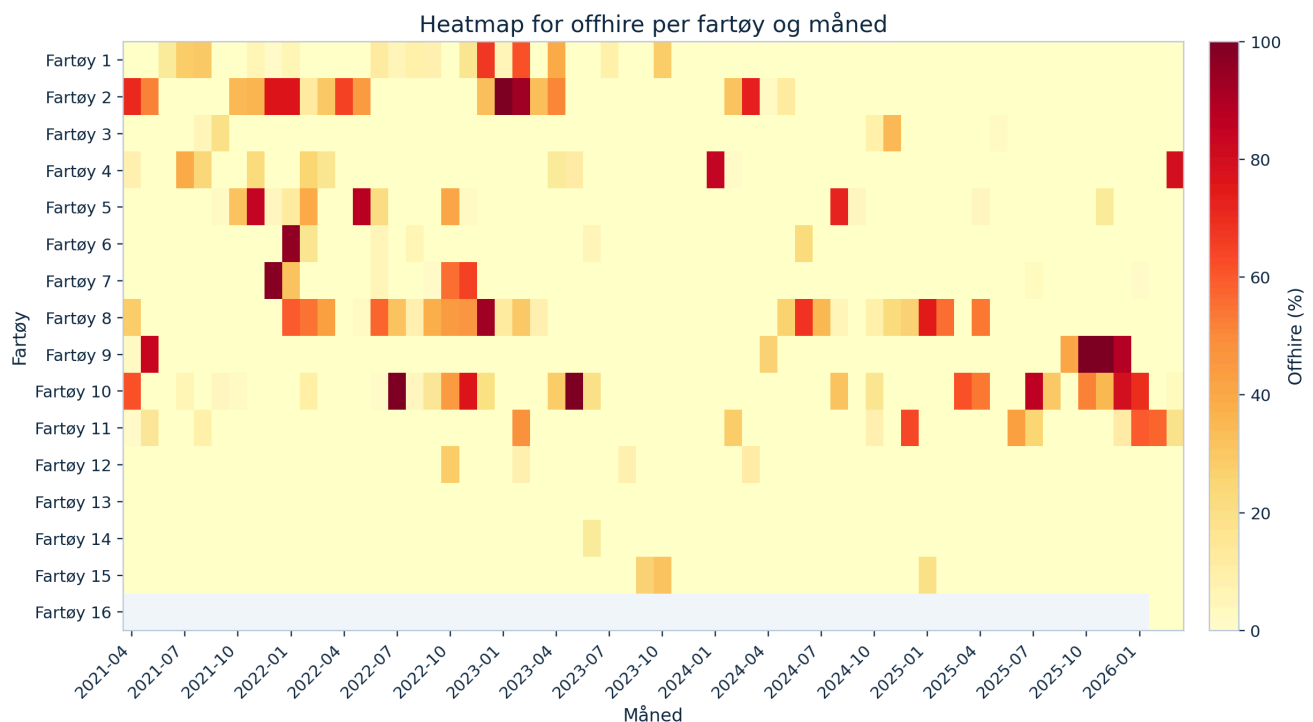
Figur 1 viser samlet offhire per måned aggregert på tvers av alle fartøy. Figuren gir et første bilde av hvor stabilt eller ujevnt materialet faktisk er over tid.



Figur 1. Samlet offhire per måned fra april 2021 til mars 2026, målt som summen av offhire i prosentpoeng på tvers av alle fartøy.

Den samlede tidsserien viser tydelige topper og rolige perioder, heller enn en jevn utvikling. Den høyeste enkeltmåneden i datasettet er januar 2022 med 283 summerte prosentpoeng, mens det også finnes to måneder der samlet offhire er null. Dette understøtter at caset ikke dreier seg om en stabil serie, men om et fenomen som opptrer episodisk og ujevnt.

Mens figur 1 viser totalnivået i materialet, viser figur 2 hvordan variasjonen fordeler seg mellom fartøyene og over tid.



Figur 2. Heatmap som viser offhire per fartøy og måned. Mørkere farger indikerer høyere offhire, mens lyse felt indikerer lave eller null registreringer.

Heatmapet viser at offhire i liten grad er jevnt fordelt mellom fartøyene. Enkelte fartøy har gjentatte og tydelige topper over flere perioder, mens andre i hovedsak har nullregistreringer. Særlig skiller **Fartøy 10**, **Fartøy 8** og **Fartøy 2** seg ut med flere markerte utslag. Figuren synliggjør også at 2021 er et oppstartår fra april og at 2026 foreløpig bare dekker årets tre første måneder. Samlet viser casebeskrivelsen dermed et konkret operasjonelt beslutningsproblem der prognostisering kan være verdifullt, men også metodisk krevende.

5.0 Metode og data

5.1 Metode

Oppgaven bruker en kvantitativ, casebasert tilnærming der historiske offhire-data analyseres for å undersøke hvordan valg av prognosemodell påvirker prediksjonsnøyaktigheten for fartøy innenfor samme offshoresegment. Simon Møkster Shipping AS brukes som casekontekst, men fartøyene er anonymisert i analysen og omtales derfor som **Fartøy 1** til **Fartøy 16**. Formålet med metoden er ikke å forklare kausale sammenhenger, men å sammenligne hvor godt ulike modeller kan predikere framtidige offhire-hendelser på grunnlag av historiske mønstre.

Studien bygger på kvantitative sekundærdata, og analyseenheten er ett fartøy i én bestemt måned. Dette gjør opplegget egnet både for tradisjonelle tidsseriemodeller og KI-baserte modeller, fordi samme datastruktur kan brukes til å sammenligne modellene på like vilkår. Arbeidet er lagt opp som en etterprøvable analyseprosess bestående av dataklargjøring, deskriptiv analyse av datasettet, eksplisitt train/test-splitt, modellering, historisk validering og fremtidsprognoser.

5.1.1 Modellutvalg og evalueringsoppsett

Valget av modeller bygger på at studien skal sammenligne to klassiske og to KI-baserte modelltradisjoner under samme betingelser. **SARIMA** og **eksponentiell glatting** representerer parsimoniske modeller som i hovedsak henter prognoseinformasjon fra seriens egen historikk og en relativt enkel struktur for nivå, sesong og avhengighet. **XGBoost** og **LSTM** representerer mer fleksible tilnærminger som kan håndtere ikke-linearitet, heterogenitet og mer komplekse mønstre, men som samtidig stiller høyere krav til feature-konstruksjon, treningsoppsett og datamengde. Ved å sammenligne disse fire modellene blir det mulig å teste om offhire-data i denne casen best beskrives av eksplisitt tidsseriedynamikk eller av mer fleksible datadrevne representasjoner.

For å sikre en rettferdig sammenligning estimeres og evalueres alle modellene på samme historiske tidsvindu og med samme ekspanderende 1-steps prognoselogikk. Dermed får ingen modell tilgang til mer fremtidsinformasjon enn de andre, og forskjeller i resultater kan i større grad tilskrives modellstruktur fremfor ulik testdesign. **MAE** brukes som hovedmål fordi metrikken er lett å tolke i samme skala som målvariabelen og mindre dominert av enkeltmåneder med svært store feil enn **RMSE**. **RMSE** brukes som støttemål fordi den tydeliggjør hvor hardt modellene straffes for store bommerter, mens **SMAPE** brukes som et prosentbasert supplement. Siden datasettet er nulltungt og inneholder flere nær-nullverdier, tolkes **SMAPE** med forsiktighet og brukes ikke som eneste grunnlag for modellrangering.

Datasettet renses først og omstruktureres til long-format før det deles i et treningssett for perioden 2021–04 til 2024–12 og et testsett for perioden 2025–01 til 2026–03. Deretter estimeres fire modeller, **SARIMA**, **Eksponentiell glatting**, **XGBoost** og **LSTM**, på historiske data og evalueres mot usette observasjoner i testperioden. Sammenligningen bygger på samme ekspanderende 1-steps evalueringslogikk for alle modellene og vurderes ved hjelp av **MAE**, **RMSE** og **SMAPE**. Etter den historiske testfasen brukes hele datasettet som grunnlag for fremtidsprognoser med horisonter på 1, 3, 6 og 12 måneder.

Denne metoden er valgt fordi den gir et transparent og sammenlignbart grunnlag for å vurdere modellvalg på samme problem og samme datagrunnlag. Ved å holde testperioden utenfor den historiske evalueringsfasen blir det mulig å vurdere modellenes generaliseringsevne, ikke bare deres tilpasning til treningsdataene. Samtidig har opplegget klare begrensninger. Datamaterialet består av sekundærdata som ikke kan verifiseres fullt ut eksternt, dataserien er relativt kort, og materialet er preget av mange nullperioder og betydelig variasjon mellom fartøyene. Funnene bør derfor tolkes som case-spesifikke for det aktuelle segmentet hos Simon Møkster Shipping AS, og ikke som direkte generaliserbare til hele offshoremarkedet.

5.2 Data

5.2.1 Datagrunnlag

Datagrunnlaget i denne studien består av kvantitative sekundærdata mottatt som et anonymisert uttrekk fra Simon Møkster Shipping AS. Datasettet er lagret som en CSV-fil og inneholder månedlige registreringer av offhire uttrykt som prosentandel dager uten kontrakt for 16 fartøy som opererer innenfor samme segment i offshorenæringen. I tillegg inneholder datasettet en tekstkolonne for spesielle behov eller krav knyttet til de enkelte fartøyene. Siden datasettet er anonymisert, omtales fartøyene i oppgaven som **Fartøy 1** til **Fartøy 16**.

Tidsperioden i datasettet strekker seg fra april 2021 til mars 2026. Materialet er organisert i seks årsblokker, én for hvert år fra 2021 til 2026, og dekker totalt 16 fartøy. Råfilen består av 125 rader inkludert årsrader, kolonneoverskrifter og tomme skillerader. Etter rensing og omstrukturering til long-format, der hver rad representerer ett fartøy i én bestemt måned, består analysegrunnlaget av 902 observasjoner. Fordelingen over tid er 135 observasjoner i 2021, 180 observasjoner per år i 2022, 2023, 2024 og 2025, samt 47 observasjoner i 2026. At 2021 og 2026 har færre observasjoner skyldes at dataserien starter i april 2021 og foreløpig bare går til og med mars 2026.

Databehandlingen har bestått av flere trinn. Først ble tomme rader, overskriftsrader og verdier markert som **N/A** fjernet fra analysegrunnlaget. Deretter ble prosentverdier standardisert til numeriske verdier, blant annet ved å omforme komma til punktum i desimaltall. Til slutt ble datasettet gjort om fra et bredt årsformat til et analyseklar long-format med variablene fartøy, måned, dato, offhire-verdi og eventuelle spesielle behov eller krav. Denne omstruktureringen var nødvendig for å kunne bruke både tradisjonelle tidsseriemodeller og maskinlæringsmodeller på samme datagrunnlag.

For selve modelleringen ble datasettet i tillegg delt i et eksplisitt trenings- og testsett. Treningsdelen dekker perioden fra april 2021 til desember 2024, mens testdelen dekker januar 2025 til mars 2026. Denne tidsbaserte splitten er valgt for å sikre at modellene evalueres på observasjoner som ligger etter treningsperioden i tid, og dermed ikke får tilgang til informasjon fra framtiden under evalueringen. I det rensede analysegrunnlaget gir dette 675 observasjoner i treningssettet og 227 observasjoner i testsettet.

Datasettet er ikke offentlig tilgjengelig, og kan derfor ikke deles fritt med leseren. Dette skyldes at materialet bygger på interne og anonymiserte virksomhetsdata. For å sikre transparens beskrives derfor variablene, tidsperioden, databehandlingen og antall observasjoner eksplisitt i oppgaven, slik at analyseopplegget kan forstås og etterprøves metodisk selv om rådataene ikke publiseres åpent.

Tabell 1 oppsummerer hovedtrekkene i datagrunnlaget. Tabellen tydeliggjør at dataserien ikke dekker hele 2021 og 2026, noe som må tas hensyn til når nivå og variasjon i materialet senere tolkes.

Tabell 1. Datadekning	Verdi
Tidsperiode	2021-04 til 2026-03

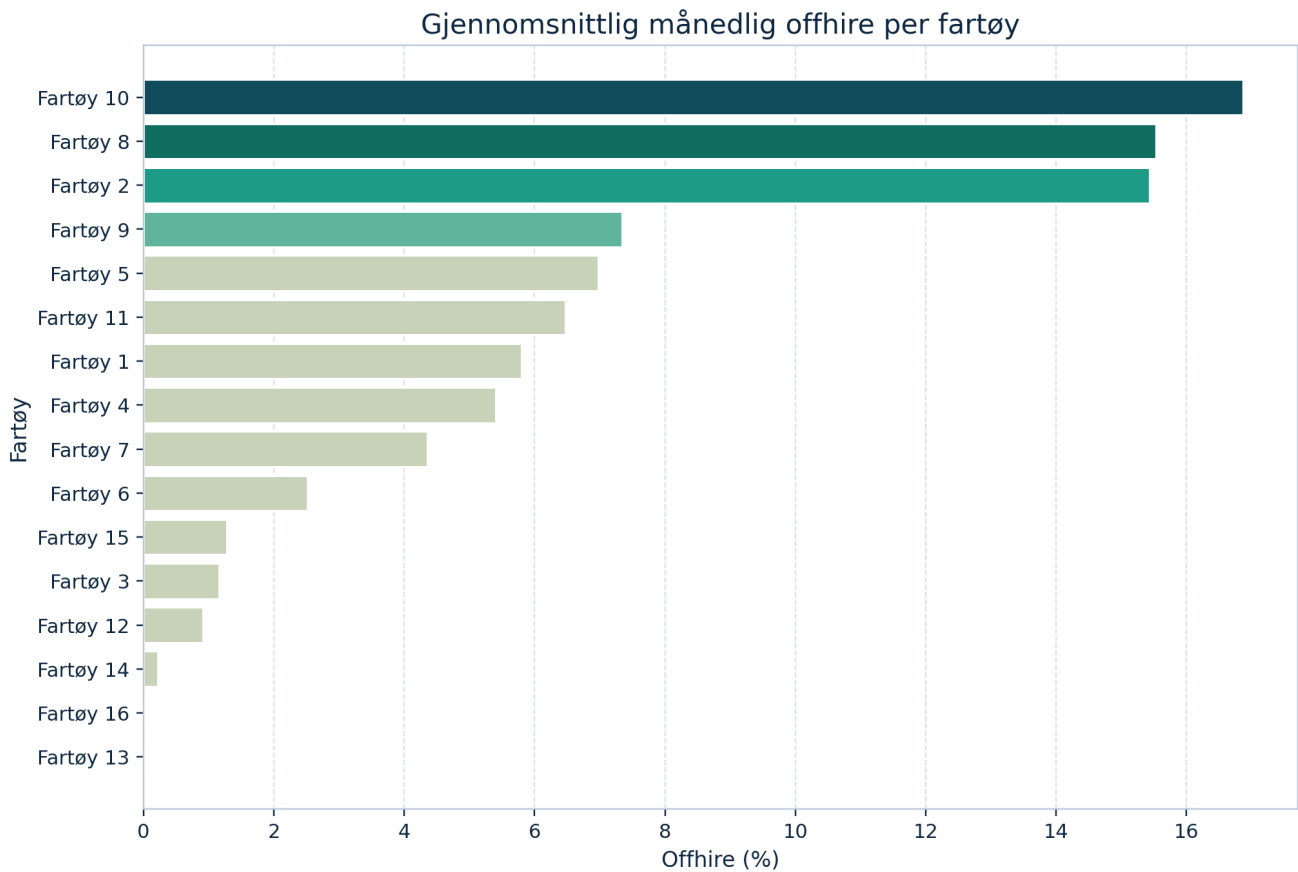
Tabell 1. Datadekning	Verdi
Antall kalendermåneder	60
Antall fartøy	16
Antall rå rader i CSV	125
Antall årsblokker	6
Antall rensede observasjoner	902
Observasjoner i 2021	135
Observasjoner i 2022	180
Observasjoner i 2023	180
Observasjoner i 2024	180
Observasjoner i 2025	180
Observasjoner i 2026	47
Kommentar	2021 starter i april, og 2026 går foreløpig bare til mars

5.2.2 Deskriptiv analyse av datasettet

De overordnede figurene i casebeskrivelsen viser at offhire varierer både over tid og mellom fartøy. I denne delen utdypes derfor datastrukturen mer detaljert for å vise hvilke trekk ved materialet som er særlig relevante for modellering. Formålet er ikke å evaluere prognosemodeller, men å beskrive datagrunnlaget på en måte som gjør det tydelig hvorfor sammenligning av ulike modelltyper er metodisk relevant.

5.2.2.1 Variasjon mellom fartøy

Figur 3 rangerer fartøyene etter gjennomsnittlig månedlig offhire i hele observasjonsperioden. Fordi 2021 og 2026 er ufullstendige år, er gjennomsnittlig månedlig nivå et mer informativt mål enn rene totalsummer.



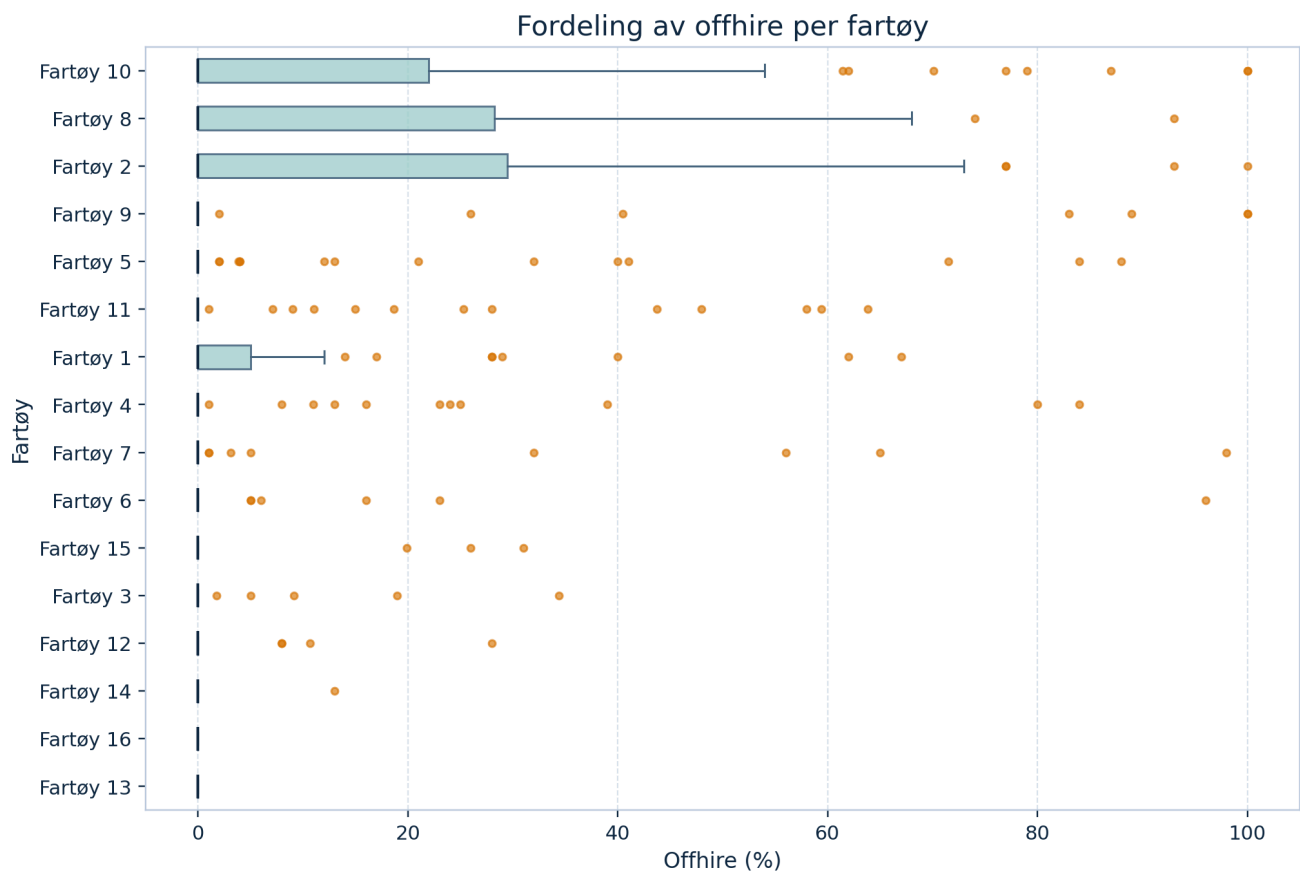
Figur 3. Gjennomsnittlig månedlig offhire per fartøy for hele perioden april 2021 til mars 2026.

Figuren viser at offhire i stor grad er konsentrert rundt et mindre antall fartøy. **Fartøy 10** har høyest gjennomsnittlig offhire med 16,86 prosent, tett fulgt av **Fartøy 8** med 15,53 prosent og **Fartøy 2** med 15,43 prosent. Samtidig viser figuren at flere fartøy ligger svært lavt gjennom hele perioden, og at **Fartøy 13** og **Fartøy 16** ikke har registrert positiv offhire i datagrunnlaget.

Tabell 2 oppsummerer de fem fartøyene med høyest gjennomsnittlig offhire og viser samtidig hvor stor andel av månedene som likevel er nullmåneder. Tabellen illustrerer at selv fartøyene med høyest nivå preges av betydelig uregelmessighet.

Tabell 2. Fartøy med høyest gjennomsnittlig offhire	Gjennomsnittlig offhire (%)	Maks offhire (%)	Andel nullmåneder (%)
Fartøy 10	16.86	100.00	56.67
Fartøy 8	15.53	93.00	58.33
Fartøy 2	15.43	100.00	68.33
Fartøy 9	7.34	100.00	88.33
Fartøy 5	6.97	88.00	76.67

Figur 4 utdyper denne variasjonen ved å vise fordelingen av offhire for hvert fartøy gjennom hele perioden, ikke bare gjennomsnittsnivået.

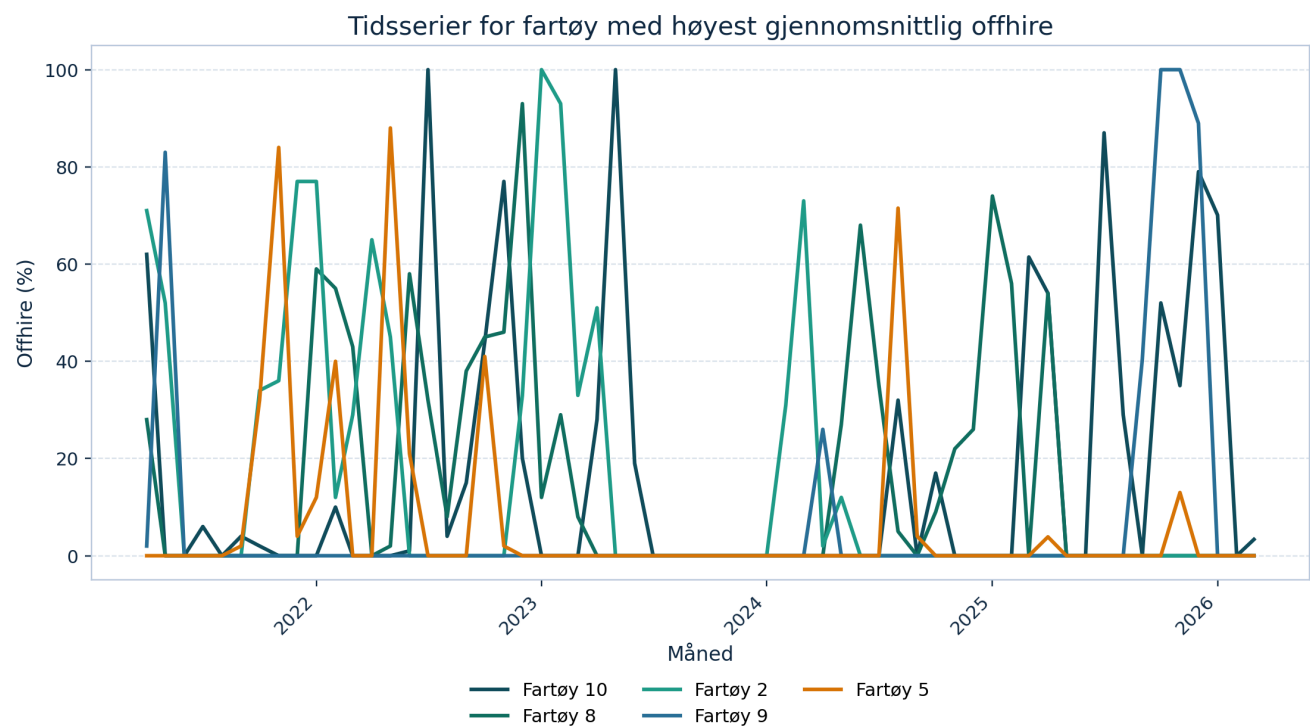


Figur 4. Boksplott som viser median, kvartiler og ekstreme observasjoner for offhire per fartøy.

Boksplottet viser at datasettet er tydelig nulltungt og høyreskjevt. For de fleste fartøy ligger medianen på eller svært nær null, mens noen få måneder trekker nivået kraftig opp. Dette betyr at gjennomsnitt alene ikke gir et fullstendig bilde av datastrukturen. Offhire fremstår i stedet som et fenomen med mange nullperioder kombinert med enkelte markerte topper, noe som er viktig å ta hensyn til i senere modellvalg.

5.2.2.2 Tidsutvikling for fartøy med høyest offhire

For å undersøke om fartøyene med høyest gjennomsnittlig offhire følger like eller ulike mønstre over tid, viser figur 5 de fem fartøyene med høyest gjennomsnittsnivå som egne tidsserier.



Figur 5. Historiske tidsserier for de fem fartøyene med høyest gjennomsnittlig månedlig offhire i datasettet.

Figuren viser at selv de mest aktive fartøyene ikke følger et jevnt eller stabilt mønster. Offhire opptrer heller i klynger eller som enkeltstående topper, og nivåene varierer betydelig mellom perioder. **Fartøy 10** og **Fartøy 8** har flere kraftige utslag gjennom perioden, mens **Fartøy 5** i større grad preges av sporadiske topper adskilt av lange perioder med null. Samlet peker den deskriptive analysen dermed mot at datasettet er preget av både sterk heterogenitet mellom fartøy og betydelig tidsmessig uregelmessighet. Dette gjør sammenligning av ulike prognosemodeller metodisk relevant.

6.0 Modellering

I denne delen bygges, testes og verifiseres modellene kun på historiske data. Selve framtidsprognosene behandles i en egen senere seksjon og inngår derfor ikke i modellbeskrivelsen nedenfor. For å gjøre modellene sammenlignbare er alle testet på samme historiske periode med samme evalueringslogikk: ekspanderende 1-steps prognoser måned for måned.

Den felles teststrukturen er oppsummert i tabell 3. **Fartøy 16** inngår ikke i hovedsammenligningen fordi fartøyet ikke har tilstrekkelig treningshistorikk før testperioden. Dermed bygger hovedsammenligningen på 15 fartøy og 225 fartøy-måneder i testsettet.

Tabell 3. Felles evalueringsoppsett	Verdi
Målvariabel	Månedlig offhire-prosent per fartøy
Treningsperiode	2021-04 til 2024-12
Testperiode	2025-01 til 2026-03
Prognosehorisont i test	1 måned per steg
Evalueringslogikk	Ekspanderende 1-steps prognose

Tabell 3. Felles evalueringsoppsett	Verdi
Sammenligningsnivå	Fartøynivå
Hovedmål	MAE
Støttemål	RMSE og sMAPE
Datagrunnlag i hovedtest	15 fartøy og 225 prediksjoner

6.1 SARIMA

Tolkning i vårt prosjekt

I denne studien brukes SARIMA fartøyvis, slik at hver tidsserie modelleres som en egen månedlig serie for offhire-prosent. Modellen skal fange opp treghet i fartøyets historiske utvikling, eventuelle sesongmønstre over året og kortsiktige avvik som ikke kan forklares av nivå alene. Den er derfor særlig egnet når neste måneds offhire kan forstås som avhengig av både tidligere måneder og gjentakende sesongstruktur.

Kort metodeformulering til oppgaven

SARIMA modellerer den månedlige offhire-serien som en kombinasjon av autoregressive ledd, differensiering, glidende gjennomsnitt og sesongkomponenter med periode 12. I denne studien estimeres modellen separat for hvert fartøy for å fange fartøyspesifikke mønstre i prosentandel dager uten kontrakt. Modellen er relevant fordi den kan representere både kortsiktig autokorrelasjon og gjentakende sesongvariasjon i månedlige tidsserier.

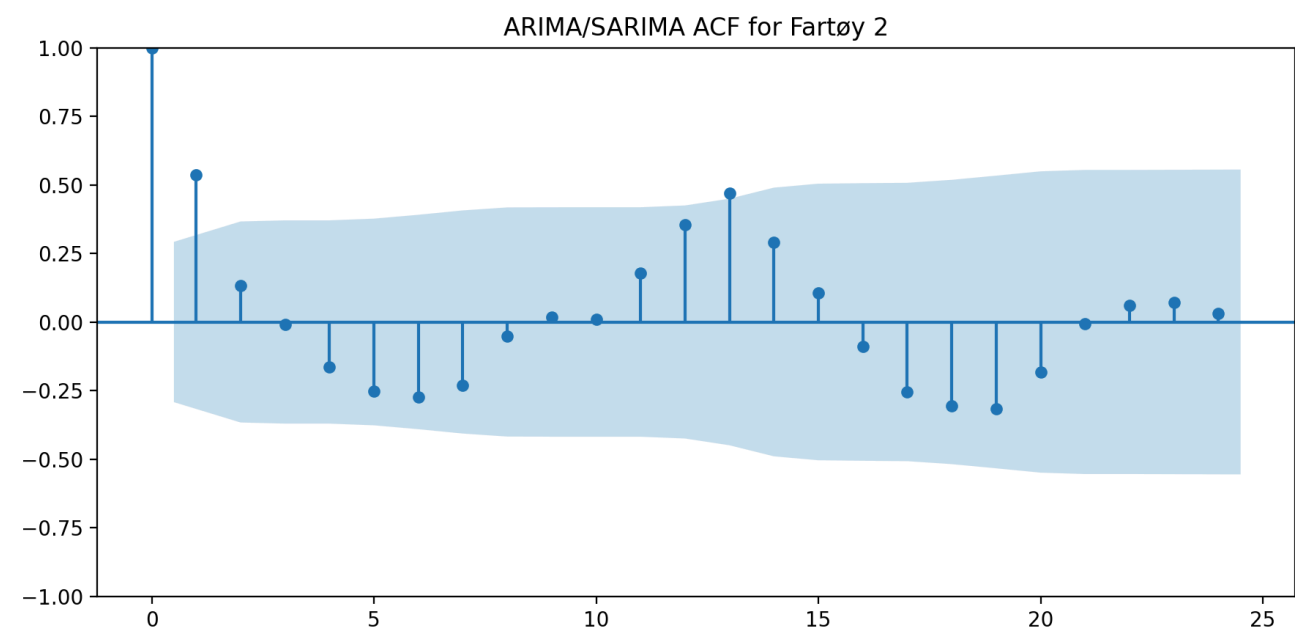
SARIMA bygger på den etablerte Box-Jenkins-tradisjonen for tidsseriemodellering, men anvendes her på fartøynivå fremfor på aggregert flåtenivå. For hvert fartøy ble det først kontrollert at tidsserien hadde tilstrekkelig historikk og variasjon. Deretter ble ADF brukt som støtte for differensieringsvalg, før et begrenset parameterrom for ARIMA/SARIMA-modeller ble estimert og rangert med AIC, BIC og parsimoni. Sesongledd med periode 12 ble tillatt der det ga mening, men ikke tvunget frem kun fordi dataene er månedlige.

Fartøy 13 hadde en konstant nullserie i treningsperioden og ble derfor håndtert som en eksplisitt konstant-baseline innen samme evalueringsramme. De øvrige 14 fartøyene fikk estimerte ARIMA/SARIMA-modeller. For et representativt fartøy med høy historisk offhire, **Fartøy 2**, viser tabell 4 de beste kandidatmodellene. Den valgte modellen for dette fartøyet ble SARIMA(2,0,0)(1,0,0,12).

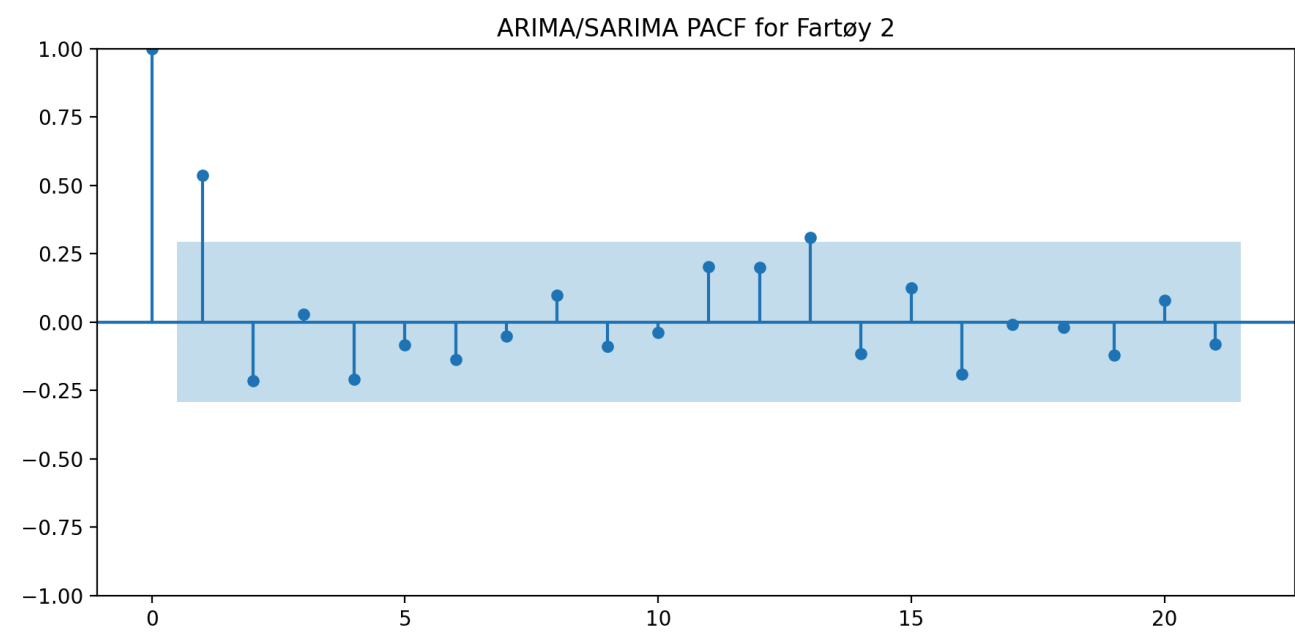
Tabell 4. Beste kandidatmodeller for representativt fartøy (Fartøy 2)	AIC	BIC
SARIMA(2,0,0)(1,0,0,12)	291.43	297.16
SARIMA(2,0,1)(1,0,0,12)	293.40	300.57
SARIMA(2,0,2)(1,0,0,12)	295.17	303.77
SARIMA(1,0,0)(1,0,0,12)	297.89	302.29

Tabell 4. Beste kandidatmodeller for representativt fartøy (Fartøy 2)	AIC	BIC
SARIMA(1,0,1) (1,0,0,12)	299.85	305.71

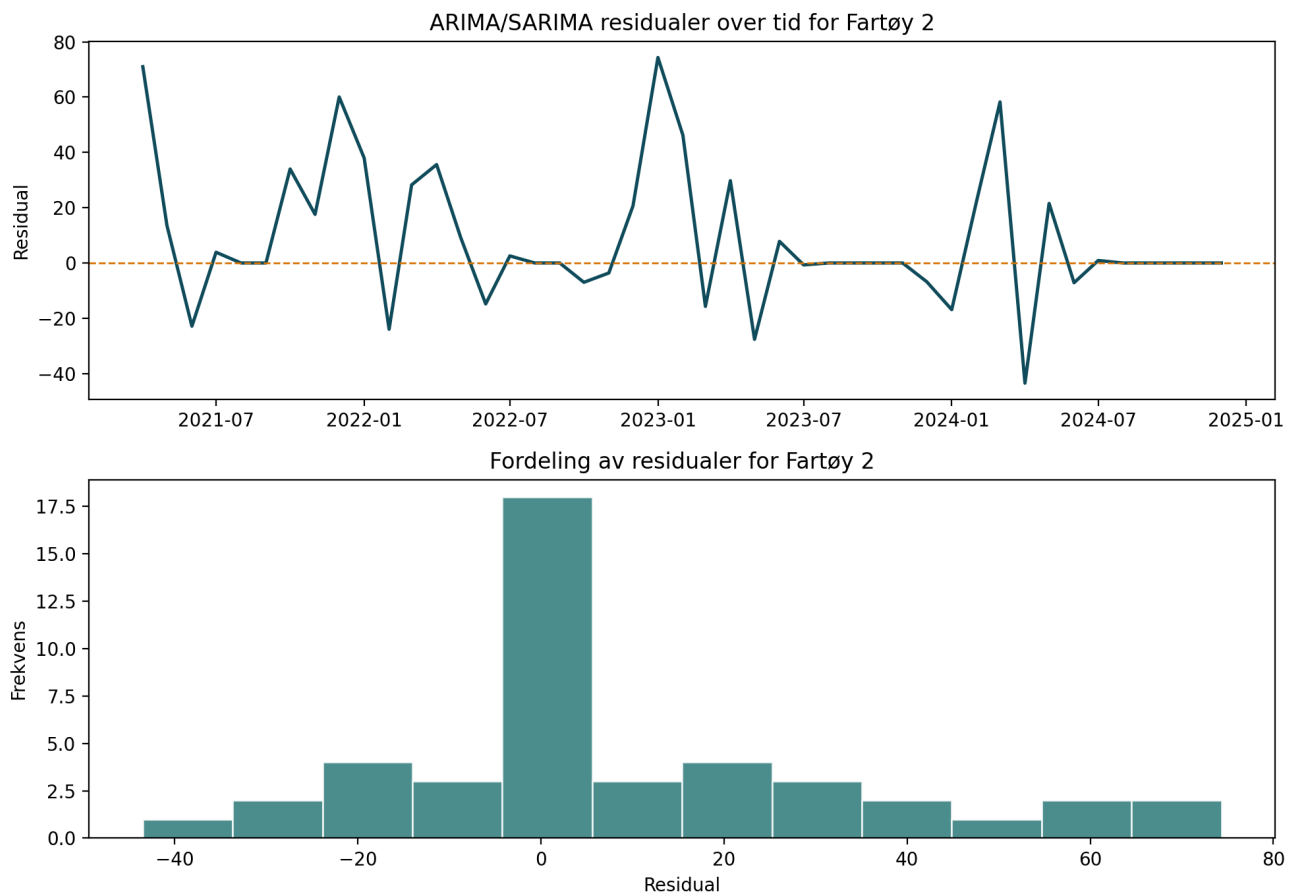
Figur 6 og 7 viser ACF og PACF for det representative fartøyet etter valgt transformasjon. Figur 8 viser residualene for samme eksempel. I tillegg viser residualtabellen i artefaktene at alle estimerte ARIMA/SARIMA-modeller hadde Ljung–Box-p-verdier over 0.05, noe som taler for at det ikke gjenstår tydelig autokorrelasjon i residualene.



Figur 6. ACF for representativt fartøy (Fartøy 2) brukt som støtte i modellidentifikasjonen.

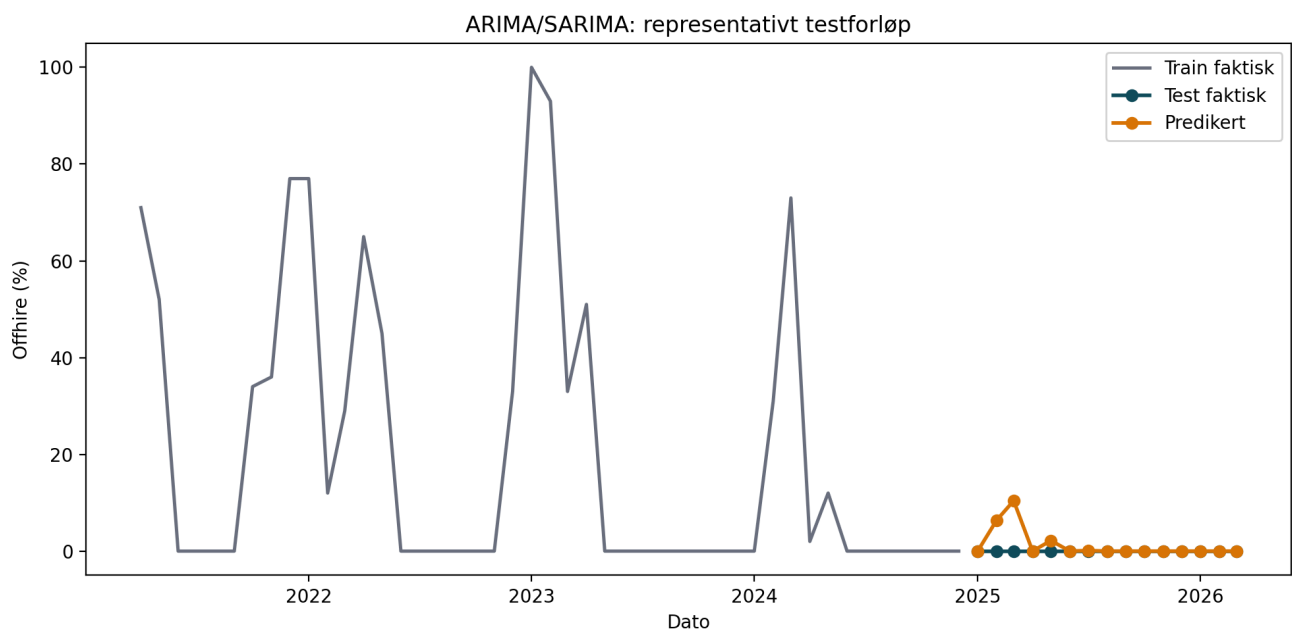


Figur 7. PACF for representativt fartøy (**Fartøy 2**) brukt som støtte i modellidentifikasjonen.



Figur 8. Residualdiagnostikk for valgt **ARIMA/SARIMA**-modell på **Fartøy 2**. Figuren viser både residualforløp og residualfordeling.

Figur 9 viser hvordan den valgte modellen treffer i testperioden for det representative fartøyet. Figuren brukes ikke som hovedbevis for modellytelsen, men som en konkret verifikasjon av at modellen faktisk følger de viktigste bevegelsene i testvinduet.



Figur 9. Historiske testprediksjoner for ARIMA/SARIMA på Fartøy 2. Grå linje viser treningsdata, blå linje faktisk testforløp og oransje linje modellens prediksjoner.

6.2 Eksponentiell glatting

Tolkning i vårt prosjekt

I denne oppgaven brukes modellen fartøyvis på månedlig offhire-prosent. Nivå, trend og sesong oppdateres fortløpende når nye månedsobservasjoner blir tilgjengelige, slik at nyere observasjoner får større vekt enn eldre observasjoner. Dette gjør modellen relevant som en konservativ, men responsiv benchmark i et datasett der flere serier har lange nullperioder avbrutt av mer uregelmessige utslag.

Kort metodeformulering til oppgaven

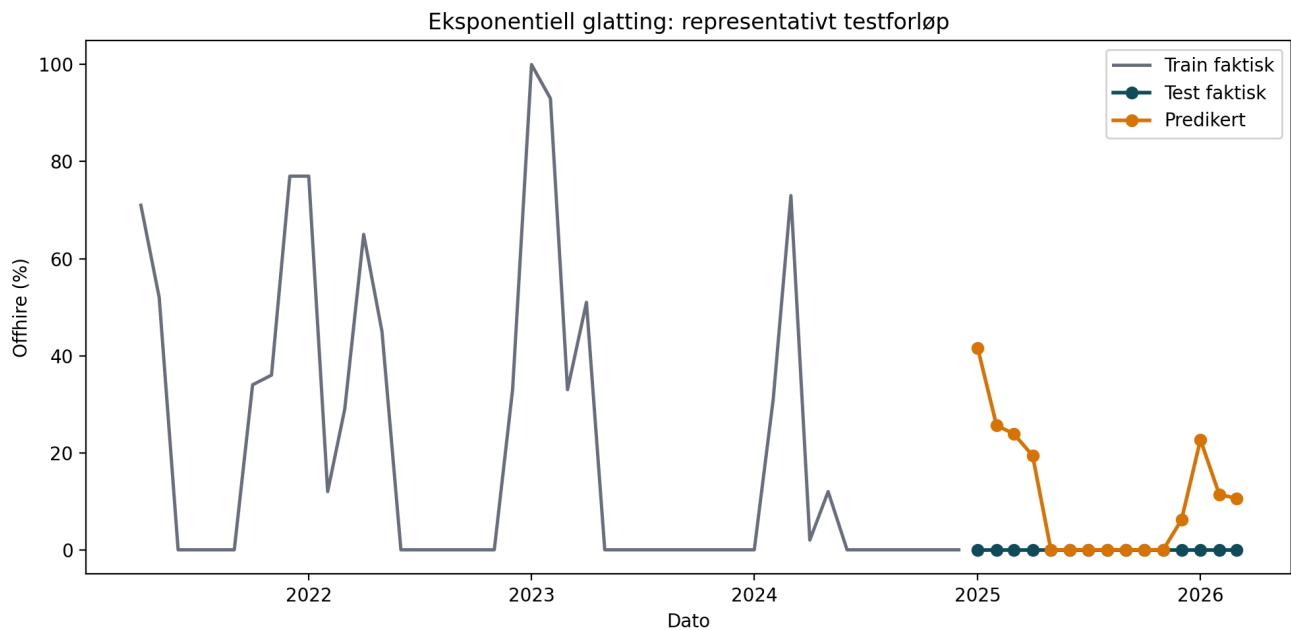
Eksponentiell glatting modellerer den månedlige offhire-serien gjennom separate komponenter for nivå, trend og sesong, der nyere observasjoner tillegges større vekt enn eldre observasjoner. I denne studien brukes modellen på fartøynivå for å estimere neste måneds prosentandel dager uten kontrakt. Modellen er relevant som en transparent benchmark fordi den krever færre strukturelle antakelser enn SARIMA, men likevel kan håndtere månedlig sesongvariasjon.

Eksponentiell glatting ble brukt som den mest konservative klassiske benchmarken. Også denne modellen ble estimert per fartøy. I stedet for å tvinge én spesifisering på alle serier ble et lite og bevisst begrenset sett av additive ETS-varianter vurdert: nivåmodell (ANN), nivå med trend (AAN) og nivå med trend og sesong (AAA). For konstante serier ble det brukt en eksplisitt konstant-baseline.

Tabell 5 oppsummerer hvilke spesifikasjoner som faktisk ble valgt. Resultatet viser at datasettet i liten grad støtter kompliserte glattemodeller: 13 fartøy endte med ANN, 1 fartøy med AAA, og 1 fartøy med konstant-baseline. Det ble ikke valgt noen AAN-modeller i siste kjøring.

Tabell 5. Valgt ETS-spesifikasjon i siste kjøring	Antall fartøy
ANN	13
AAA	1
CONST	1

Residualdiagnostikken viser at ETS fungerer rimelig godt for mange fartøy, men svakere enn ARIMA/SARIMA på enkelte serier. Særlig Fartøy 2 og Fartøy 7 fikk Ljung-Box-p-verdier under 0.05, noe som indikerer at restautokorrelasjon ikke var like godt håndtert i alle tilfeller. Figur 10 viser testforløpet for det representative fartøyet.



Figur 10. Historiske testprediksjoner for eksponentiell glatting på *Fartøy 2*. Figuren viser at modellen fanger nivået i serien, men håndterer topper svakere enn den beste ARIMA/SARIMA-modellen.

6.3 XGBoost

Tolkning i vårt prosjekt

I denne studien er **XGBoost** ikke en klassisk univariat tidsseriemodell, men en feature-basert panelmodell på fartøy-måned-nivå. Feature-vektoren x_i består av laggede observasjoner, rullerende gjennomsnitt, rullerende standardavvik, kalenderkomponenter og fartøyspesifikke trekk. Modellen predikerer dermed neste måneds offhire-prosent ved å lære mønstre i konstruerte tidsserie-features, heller enn å spesifisere tidsavhengigheten eksplisitt i én serieformel.

Kort metodeformulering til oppgaven

XGBoost modellerer prognoseproblemet som en supervisert regresjonsoppgave der prediksjonen uttrykkes som summen av flere beslutningstrær. I denne studien brukes modellen på fartøy-måned-paneldata, der neste måneds offhire-prosent predikeres fra laggede observasjoner, rullerende statistikk, kalenderinformasjon og fartøyspesifikke kjennetegn. Modellen er relevant fordi den kan fange ikke-lineære sammenhenger og interaksjoner uten å være bundet til en eksplisitt univariat tidsseriemodell.

XGBoost ble satt opp som én global modell på fartøy-måned-paneldata. Modellen fikk et eksplisitt feature-set som bare brukte informasjon tilgjengelig før hver testmåned. Dermed følger også denne modellen samme ekspanderende 1-steps logikk som de klassiske modellene.

Feature-settet er oppsummert i tabell 6. Poenget var å gi modellen både kortsiktig historikk, glattet historikk og kalenderinformasjon, samtidig som fartøyspesifikke forskjeller kunne fanges gjennom `vessel` og `special_flag`.

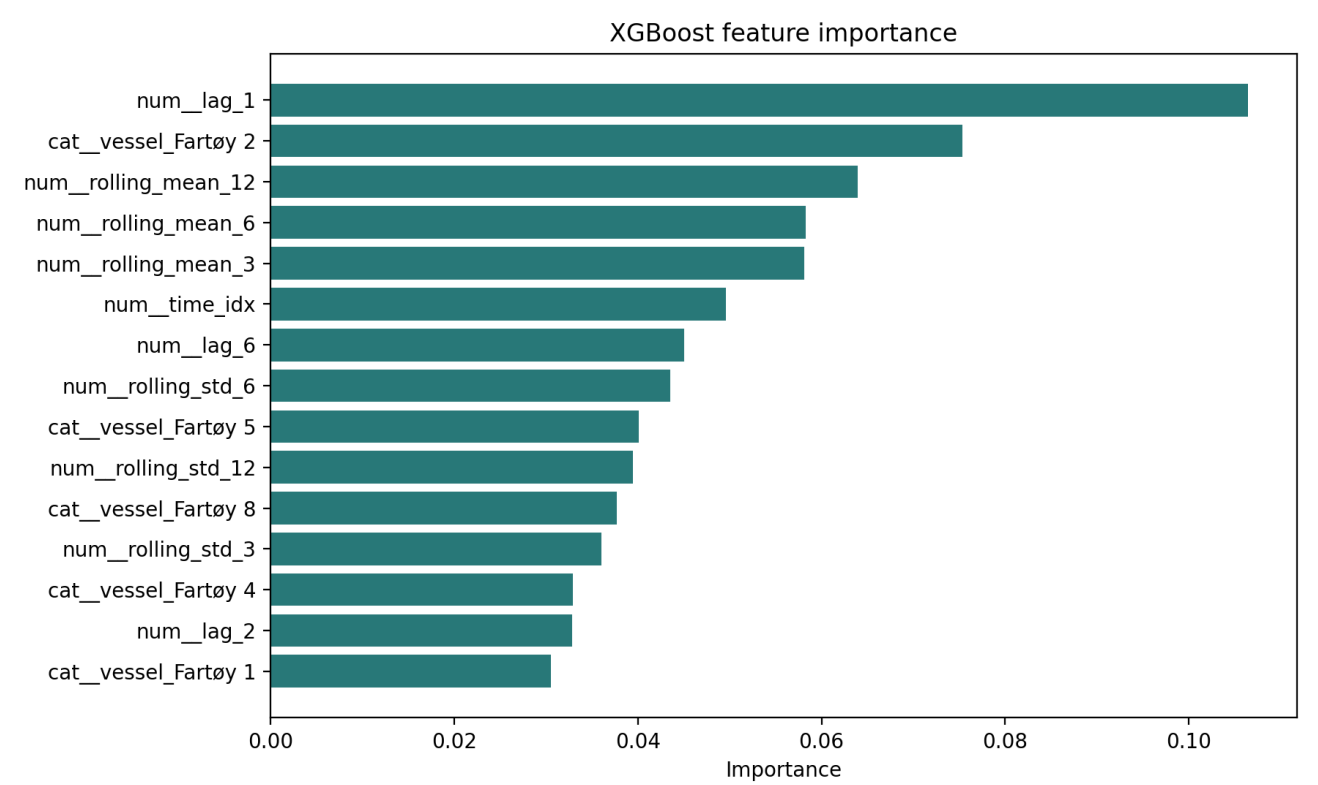
Tabell 6. XGBoost-featuregrupper	Innhold
Historiske lags	lag_1, lag_2, lag_3, lag_6, lag_12

Tabell 6. XGBoost-featuregrupper	Innhold
Rullerende nivå	rolling_mean_3, rolling_mean_6, rolling_mean_12
Rullerende variasjon	rolling_std_3, rolling_std_6, rolling_std_12
Kalender	month_num, quarter_num, year_num, time_idx, month_sin, month_cos
Kategoriske trekk	vessel, special_flag

Hyperparametrene ble holdt faste gjennom hele testoppsettet, som vist i tabell 7.

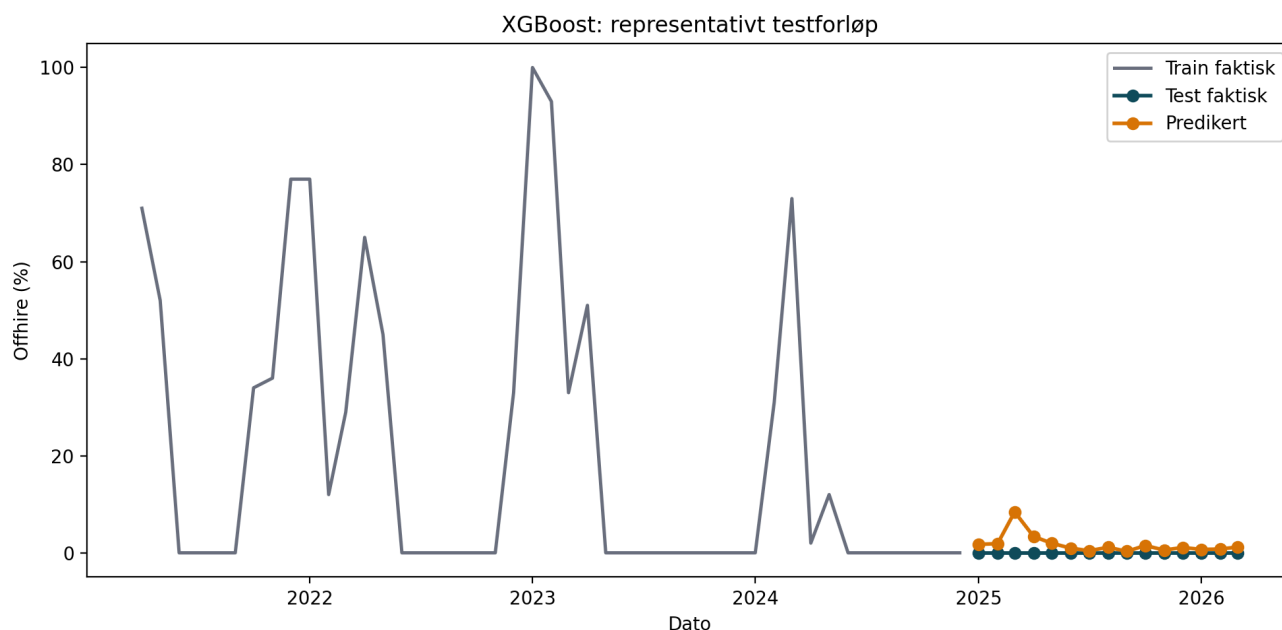
Tabell 7. XGBoost-hyperparametre	Verdi
n_estimators	200
max_depth	4
learning_rate	0.05
subsample	0.90
colsample_bytree	0.90

Figur 11 viser at modellen i hovedsak bygger på kort og mellomlang historikk. lag_1 er viktigst, men også rolling_mean_12, rolling_mean_6, rolling_mean_3 og enkelte fartøyindikatorer bidrar mye. Dette er konsistent med at problemet både har tidsseriepreg og tydelig fartøyheterogenitet.



Figur 11. Viktigste features i referansemodellen for **XGBoost** estimert på treningsperioden. Laggede verdier og rullerende gjennomsnitt dominerer.

Figur 12 viser den historiske testytelsen for det representative fartøyet. Sammenlignet med de klassiske modellene framstår **XGBoost** som mer fleksibel, men fortsatt sårbar i perioder med svært uregelmessige toppe.



Figur 12. Historiske testprediksjoner for **XGBoost** på **Fartøy 2**. Figuren viser modellens evne til å følge nivåendringer uten eksplisitt tidsseriemodell.

6.4 LSTM

Tolkning i vårt prosjekt

I denne studien brukes **LSTM** som en global sekvensmodell på fartøy-måned-data, der et observasjonsvindu på 12 måneder brukes for å predikere neste måned. Inputvektoren x_t inneholder den historiske målvariabelen samt kalender- og fartøyrelatert informasjon, slik at modellen kan lære tidsavhengigheter uten at disse må spesifiseres manuelt. Den skjulte tilstanden h_t representerer kortsiktig informasjon fra sekvensen, mens celletilstanden c_t fungerer som modellens mer langvarige hukommelse.

Kort metodeformulering til oppgaven

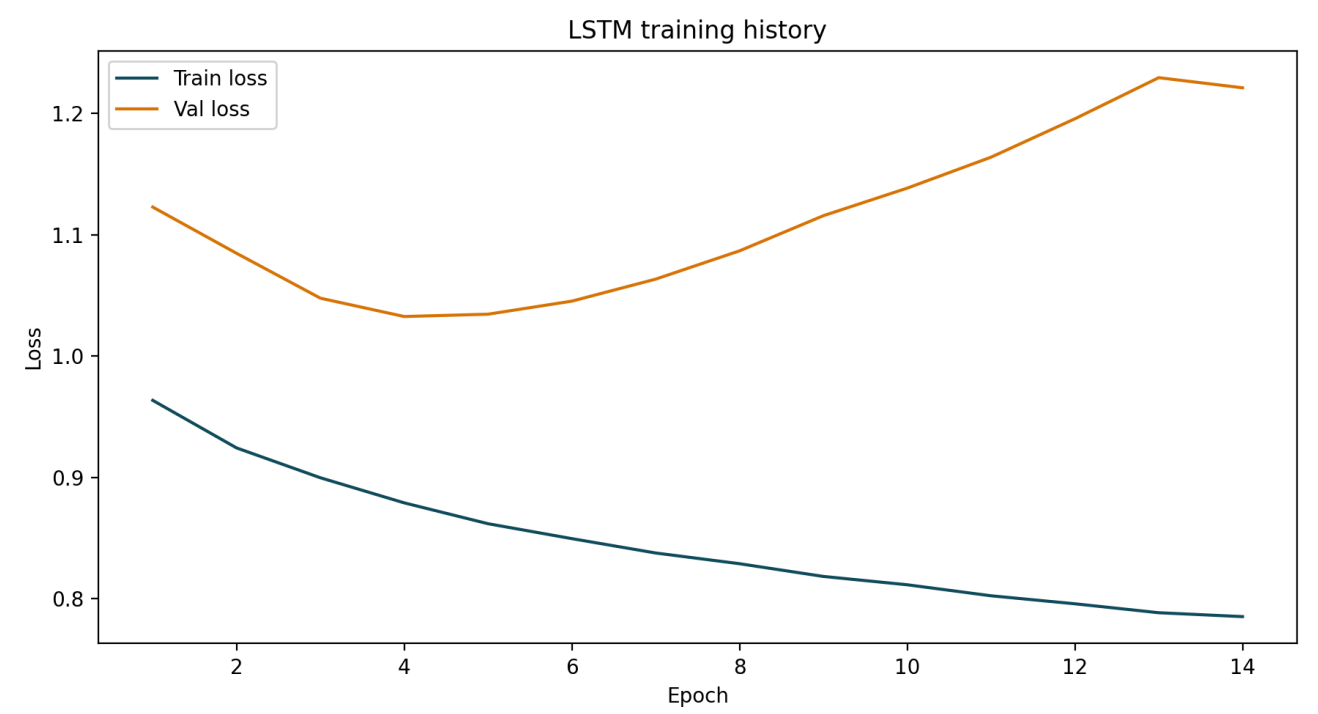
LSTM er en rekurrent sekvensmodell som lærer tidsavhengigheter gjennom en intern hukommelsesstruktur bestående av skjult tilstand og celletilstand. I denne studien brukes modellen på sekvenser av månedlige observasjoner for å predikere neste måneds offhire-prosent for fartøyene i datasettet. Modellen er relevant fordi den kan lære mønstre over flere måneder uten at tidsavhengigheten må spesifiseres eksplisitt på forhånd.

LSTM ble bygget som én global sekvensmodell på fartøy-måned-data. Hver observasjon ble representert som en sekvens på 12 måneder, med fire inputfeatures per tidssteg: den historiske målvariabelen (`offhire_days` i kodegrunnlaget), `month_sin`, `month_cos` og `special_flag`. All skalering ble estimert på treningsdata. Modellen ble deretter re-trent måned for måned i samme ekspanderende testoppsett som de øvrige modellene.

Det konkrete oppsettet er vist i tabell 8.

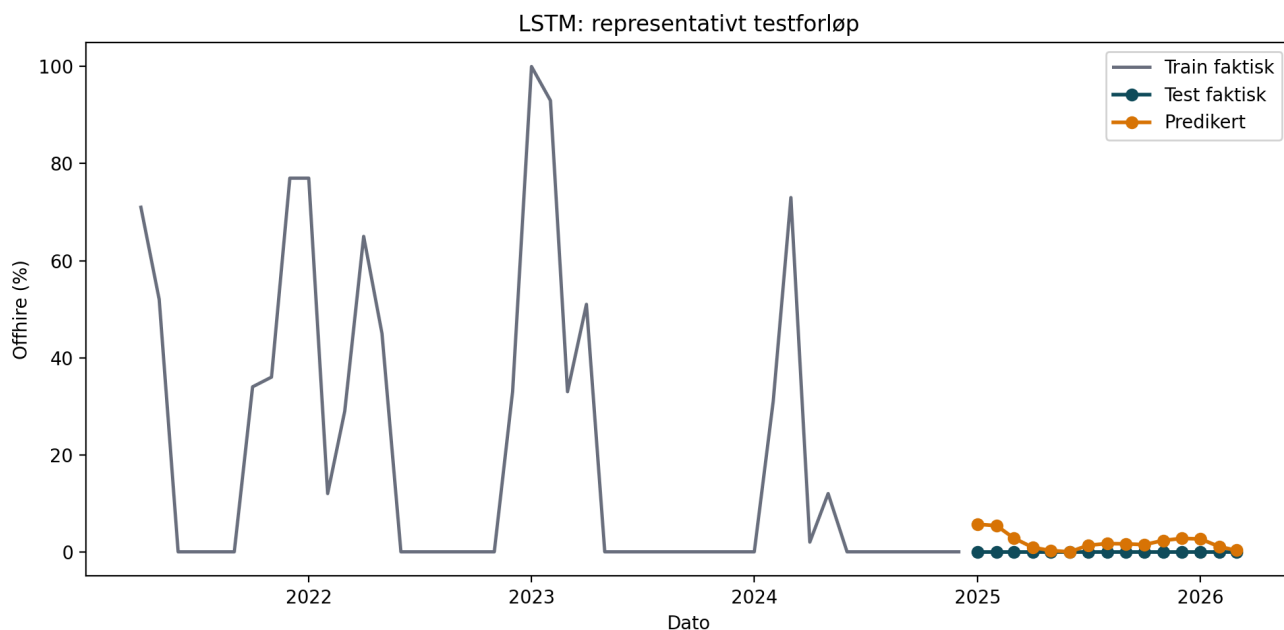
Tabell 8. LSTM-oppsett i siste kjøring	Verdi
Sekvenslengde	12 måneder
Inputformat	samples x timesteps x features
Inputfeatures	offhire_days, month_sin, month_cos, special_flag
LSTM-enheter	32
Dense-enheter	16
Batch size	8
Maks antall epoker	100
Tidlig stopping	Ja, med gjenoppretting av beste vektorer

Figur 13 viser treningshistorikken fra referansekjøringen på treningsperioden. Valideringstapet flater tidlig ut og begynner deretter å stige, noe som understøtter at tidlig stopping er nødvendig for å unngå overtilpasning.



Figur 13. Trenings- og valideringstap for LSTM estimert på treningsperioden. Figuren viser at modellen lærer raskt, men at valideringstapet ikke forbedres videre etter de første epokene.

Figur 14 viser testforløpet for det representative fartøyet. Som for XGBoost er modellen fleksibel, men den store fordelene over de beste klassiske modellene er ikke tydelig i dette datasettet.



Figur 14. Historiske testprediksjoner for LSTM på Fartøy 2. Figuren viser at modellen følger nivåendringer relativt godt, men ikke tydelig bedre enn de sterkeste alternativene.

Samlet viser modelleringskapitlet at alle fire modellfamiliene nå er bygget og testet innenfor samme historiske evalueringsramme. Forskjellen mellom modellene ligger derfor ikke lenger i oppsettet, men i hvordan de håndterer samme datastruktur under samme betingelser.

6.5 Oppsett for fremtidsprognoser

Etter at modellene var evaluert på den historiske testperioden, ble fremtidsprognosene kjørt som en egen fase. I denne fasen ble hele datasettet til og med 2026-03 brukt som treningsgrunnlag, og første prognosemåned ble dermed 2026-04. For alle fire modellene ble det generert prognoser 1, 3, 6 og 12 måneder fram i tid.

For de klassiske modellene ble prognosene laget per fartøy med fler-stegsprognoser direkte fra den estimerte modellen. For XGBoost og LSTM ble prognosene generert iterativt måned for måned, slik at predikert verdi fra ett steg inngår som historisk input i neste steg. Dette gjør at alle modellene kan sammenlignes på samme framtidige datovindu, samtidig som de beholder sin opprinnelige modellstruktur.

Fremtidsprognosene evalueres ikke med MAE, RMSE eller SMAPE, fordi faktiske observasjoner ikke finnes ennå. I stedet brukes de som modellbaserte scenariobeskrivelser. For å gjøre resultatene sporbare ble det lagret egne forecast-filer både samlet og per horisont, samt figurer som summerer forventet offhire per måned og modell.

7.0 Resultater

Resultatdelen er delt i to. Først presenteres resultatene fra den historiske modelltesten, som viser hvordan modellene presterer på kjente holdout-data. Deretter presenteres fremtidsprognosene for 1, 3, 6 og 12 måneder fram i tid. Denne todelingen er viktig fordi historisk test kan evalueres med feilmetriker, mens fremtidsprognoser bare kan tolkes som modellbaserte estimer.

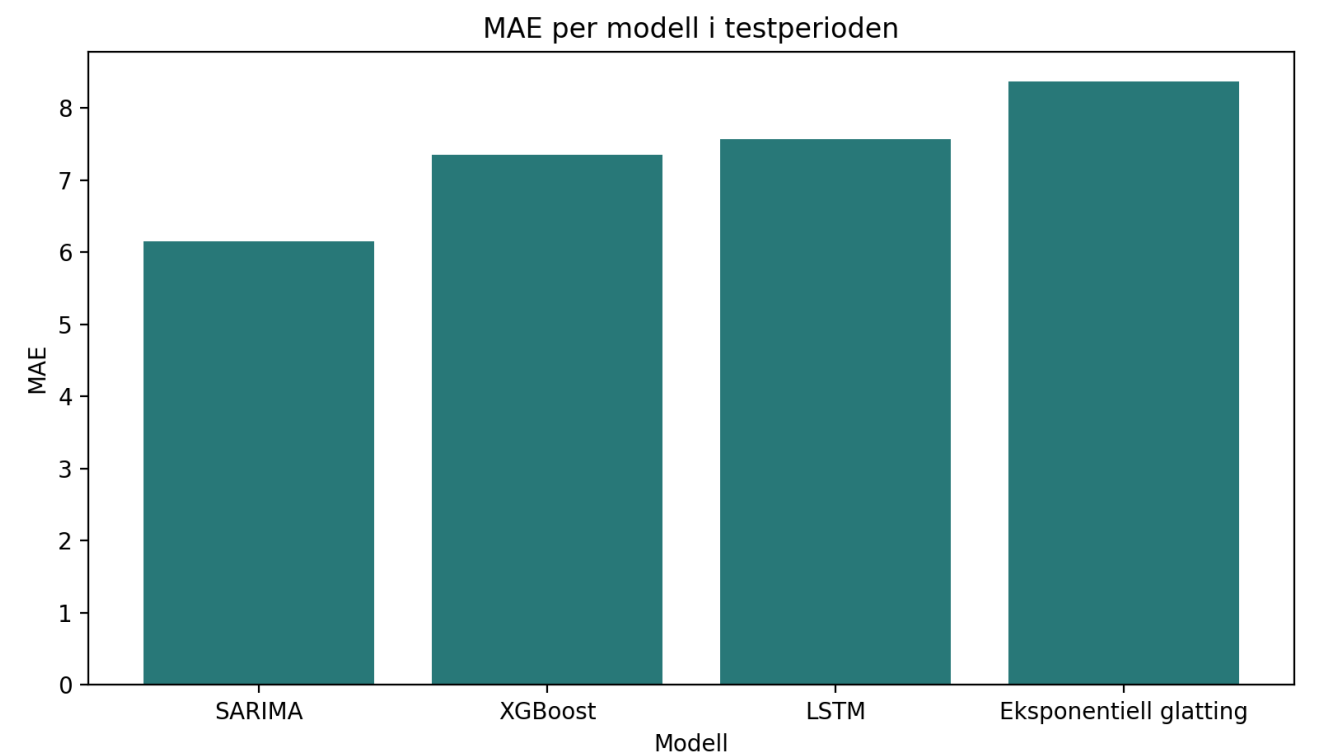
7.1 Resultater fra historisk modelltesting

Alle modeller er evaluert på de samme 225 fartøy-månedene i testperioden fra januar 2025 til mars 2026. MAE brukes som hovedmål, mens RMSE og sMAPE brukes som støttemål. Siden datasettet er svært nulltungt, må sMAPE tolkes med forsiktighet; metrikken blir høy når både faktiske og predikerte verdier ligger nær null.

Tabell 9 viser det samlede testresultatet. ARIMA/SARIMA oppnår lavest MAE og lavest RMSE i siste kjøring, mens XGBoost og LSTM ligger svært nær hverandre. Eksponentiell glatting er svakest av de fire når alle sammenlignes på samme fartøynivå og samme evalueringslogikk.

Tabell 9. Samlet testresultat for modellene	Antall prediksjoner	MAE	RMSE	sMAPE
ARIMA/SARIMA	225	6.15	16.78	100.44
XGBoost	225	7.35	17.40	182.98
LSTM	225	7.57	17.41	178.77
Eksponentiell glatting	225	8.37	17.95	168.62

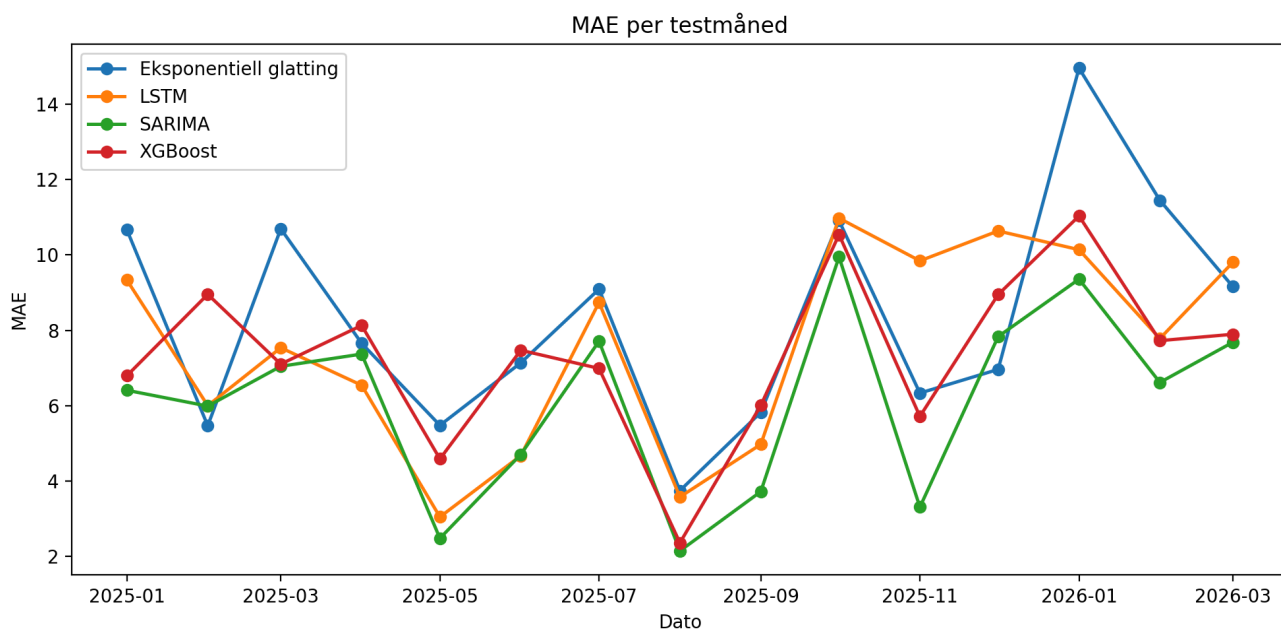
Figur 15 visualiserer de samme MAE-resultatene som en samlet sammenligning. Figuren tydeliggjør at forskjellen mellom de tre beste modellene er relativt liten, men at ARIMA/SARIMA likevel kommer best ut i siste kjøring.



Figur 15. Samlet MAE for de fire modellene i testperioden. Lavere verdi indikerer bedre prediksjonsnøyaktighet.

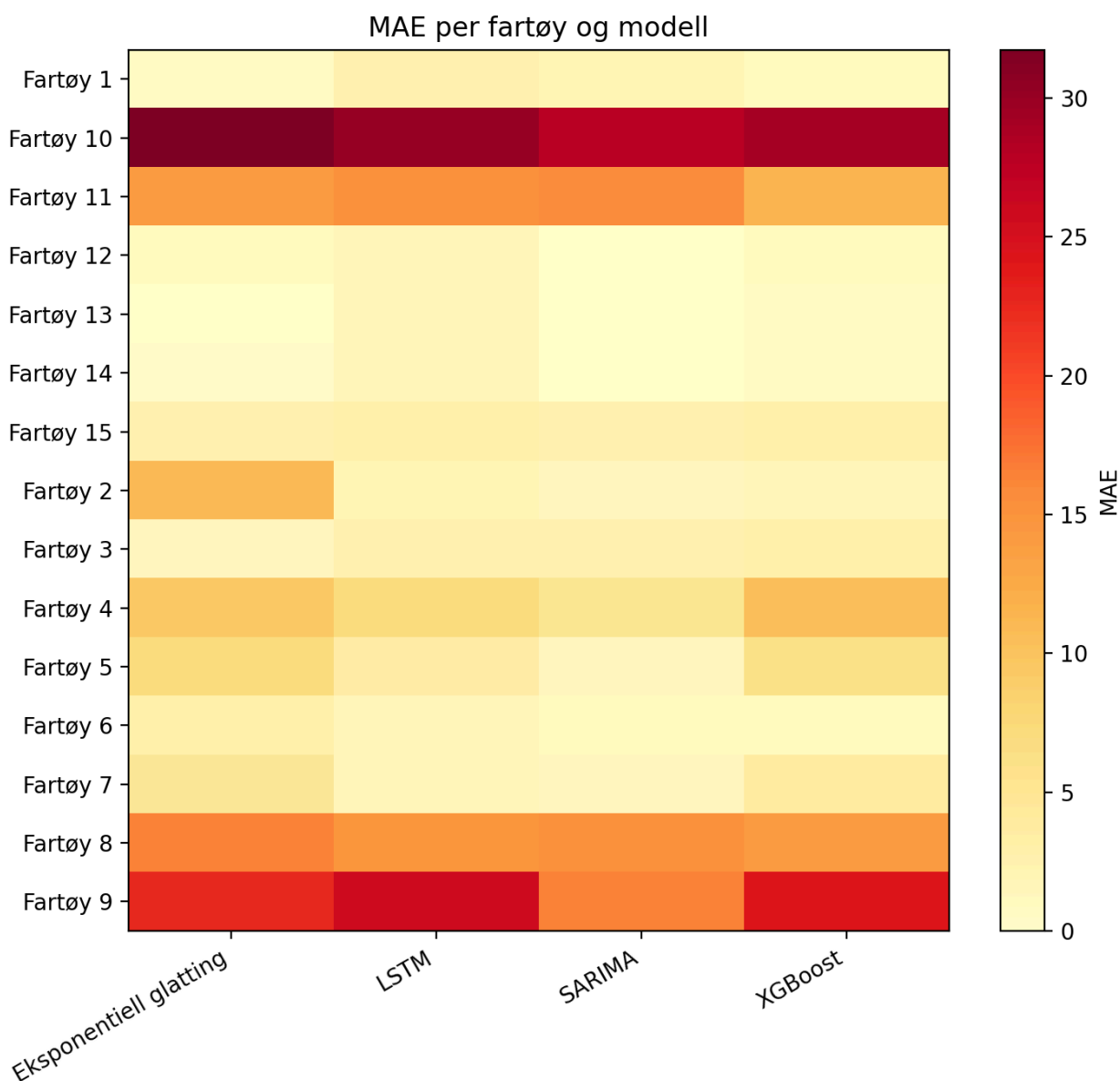
Figur 16 viser hvordan MAE varierer mellom testmånedene. Ingen modell dominerer alle måneder fullstendig, men ARIMA/SARIMA er gjennomgående sterk og særlig stabil i flere av månedene med mer moderate nivåer. Samtidig viser figuren at alle modellene får høyere feil i måneder der offhire-nivået er preget av store hopp og

episoder.



Figur 16. MAE per måned i testperioden for de fire modellene. Figuren viser hvordan modellytelsen varierer over tid, ikke bare samlet.

Figur 17 viser MAE per fartøy og modell som heatmap. Figuren tydeliggjør at de største feilene er konsentrert rundt noen få fartøy, særlig **Fartøy 10**, **Fartøy 9** og **Fartøy 8**, mens flere fartøy med lav eller null offhire er enklere å predikere for alle modellene. Dette betyr at samlet modellrangering i stor grad påvirkes av hvor godt modellene håndterer de mest krevende fartøyene.



Figur 17. Heatmap som viser MAE per fartøy og modell i testperioden. Mørkere felt indikerer høyere prediksjonsfeil.

Resultatene fra den historiske modelltesten peker mot tre hovedobservasjoner. For det første fungerer klassiske tidsseriemodeller fortsatt svært godt i dette datasettet, særlig når ARIMA/SARIMA får modelleres per fartøy og verifiseres med residualdiagnostikk. For det andre presterer XGBoost og LSTM konkurransedyktig, men uten å gi en tydelig gevinst over den beste klassiske modellen. For det tredje er forskjellene mellom modellene mindre enn forskjellene mellom fartøyene, noe som understreker at problemet er like mye et spørsmål om datakarakter som om modellvalg.

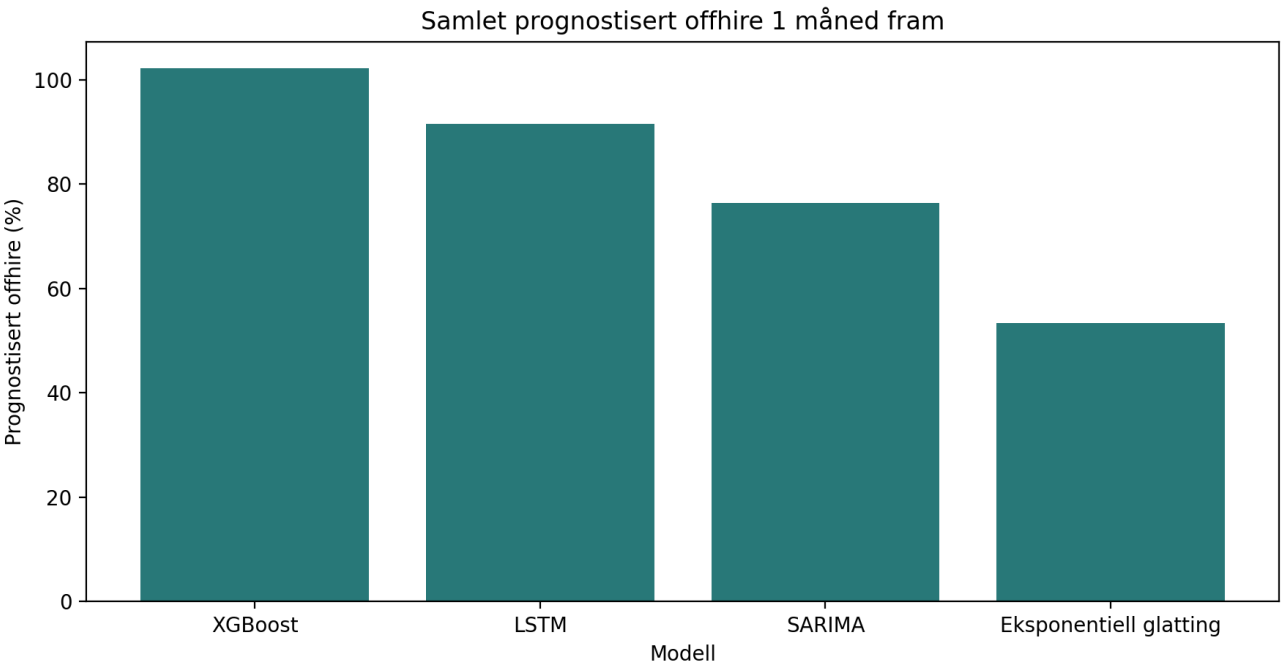
7.2 Resultater fra fremtidsprognoser

Etter at modellene var testet historisk, ble alle fire modellene kjørt på hele historikken til og med 2026-03. Prognosevinduet dekker dermed perioden 2026-04 til 2027-03. For å holde hovedteksten lesbar presenteres tabellene nedenfor som samlet prognostisert offhire per måned på tvers av de 15 fartøyene som fikk framtidsprognoser. De detaljerte fartøyvise forecast-tabellene er lagret som egne artefakter i 004 data/modeling/outputs/shared/.

7.2.1 Prognose 1 måned fram

Tabell 10 viser énmånedersprognosen for april 2026. Allerede på dette korte nivået er det tydelig at modellene ikke er helt samstemte. **XGBoost** gir høyest samlet prognose med **102.19**, mens eksponentiell glatting ligger lavest med **53.37**. På fartøynivå peker tre av fire modeller sterkest mot **Fartøy 10**, mens **ARIMA/SARIMA** har den høyeste enkeltprognosen på **Fartøy 9** med **42.93**.

Tabell 10. Samlet prognostisert offhire 1 måned fram	Eksponentiell glatting	LSTM	ARIMA/SARIMA	XGBoost
2026-04	53.37	91.60	76.35	102.19



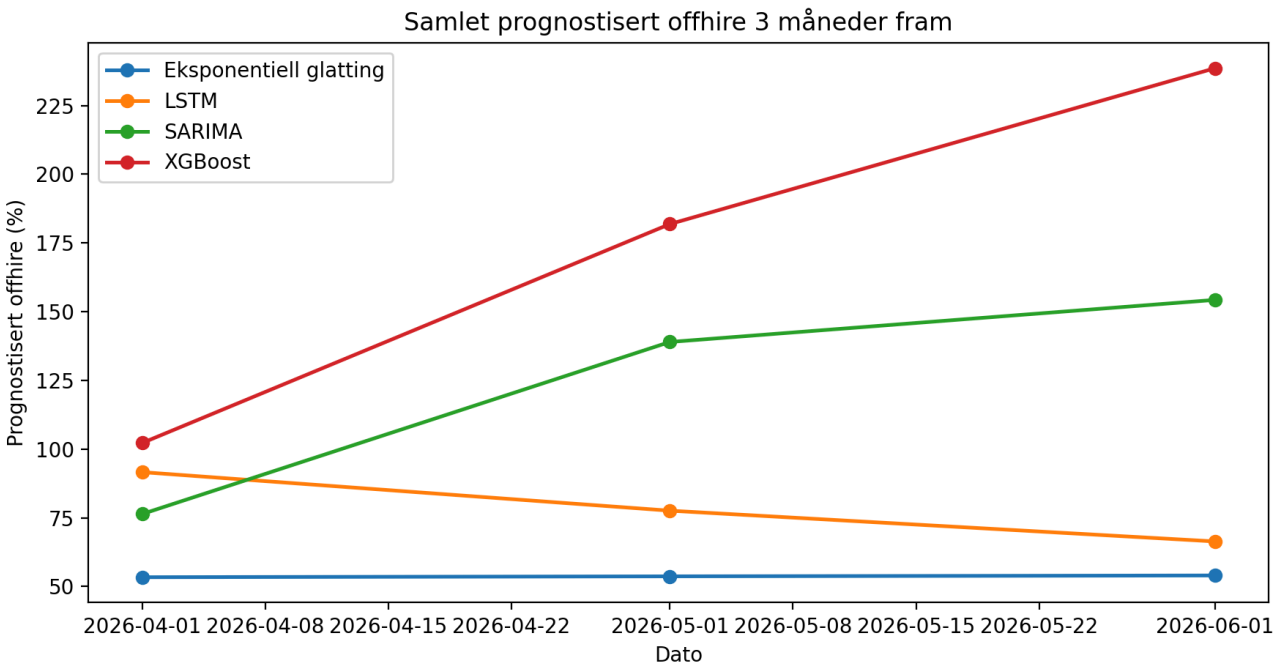
Figur 18. Samlet prognostisert offhire i april 2026 for de fire modellene.

7.2.2 Prognose 3 måneder fram

Tabell 11 viser at forskjellene øker raskt når horisonten forlenges til tre måneder. **Eksponentiell glatting** ligger nærmest flatt gjennom hele vinduet, mens **LSTM** beveger seg moderat nedover. **ARIMA/SARIMA** og særlig **XGBoost** estimerer langt høyere nivåer i mai og juni. Ved utgangen av juni 2026 er forskjellen mellom høyeste og laveste modell over **180** prognostiserte offhire-enheter.

Tabell 11. Samlet prognostisert offhire 3 måneder fram	Eksponentiell glatting	LSTM	ARIMA/SARIMA	XGBoost
2026-04	53.37	91.60	76.35	102.19
2026-05	53.69	77.58	138.96	181.85

Tabell 11. Samlet prognostisert offhire 3 måneder fram	Eksponentiell glatting	LSTM	ARIMA/SARIMA	XGBoost
2026-06	54.00	66.39	154.27	238.57

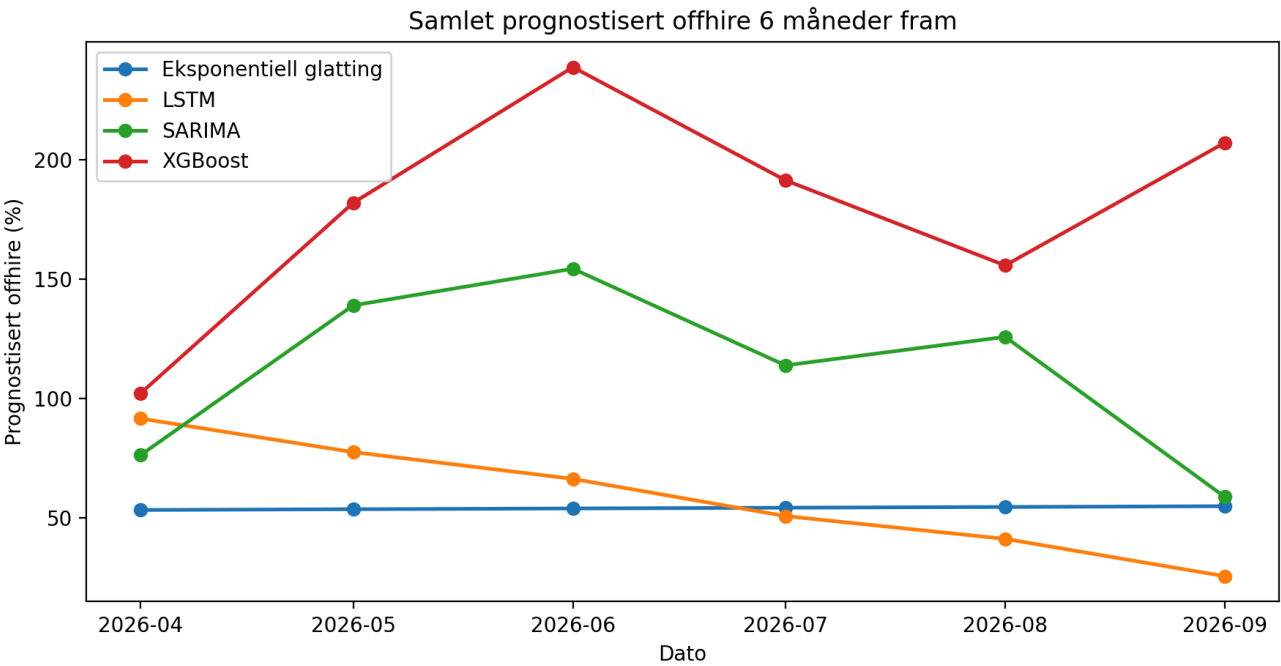


Figur 19. Samlet prognostisert offhire fra april til juni 2026 for de fire modellene.

7.2.3 Prognose 6 måneder fram

Tabell 12 viser seksmånedersprognosen fra april til september 2026. Også her fremstår eksponentiell glatting som den mest konservative modellen, med et nesten uendret totalnivå fra måned til måned. LSTM faller tydelig utover sommeren, mens ARIMA/SARIMA varierer mer og beholder flere markerte topper. XGBoost ligger gjennomgående høyest og holder seg over 150 i alle måneder unntatt april.

Tabell 12. Samlet prognostisert offhire 6 måneder fram	Eksponentiell glatting	LSTM	ARIMA/SARIMA	XGBoost
2026-04	53.37	91.60	76.35	102.19
2026-05	53.69	77.58	138.96	181.85
2026-06	54.00	66.39	154.27	238.57
2026-07	54.32	50.80	113.85	191.26
2026-08	54.63	41.29	125.77	155.66
2026-09	54.95	25.67	58.83	206.98



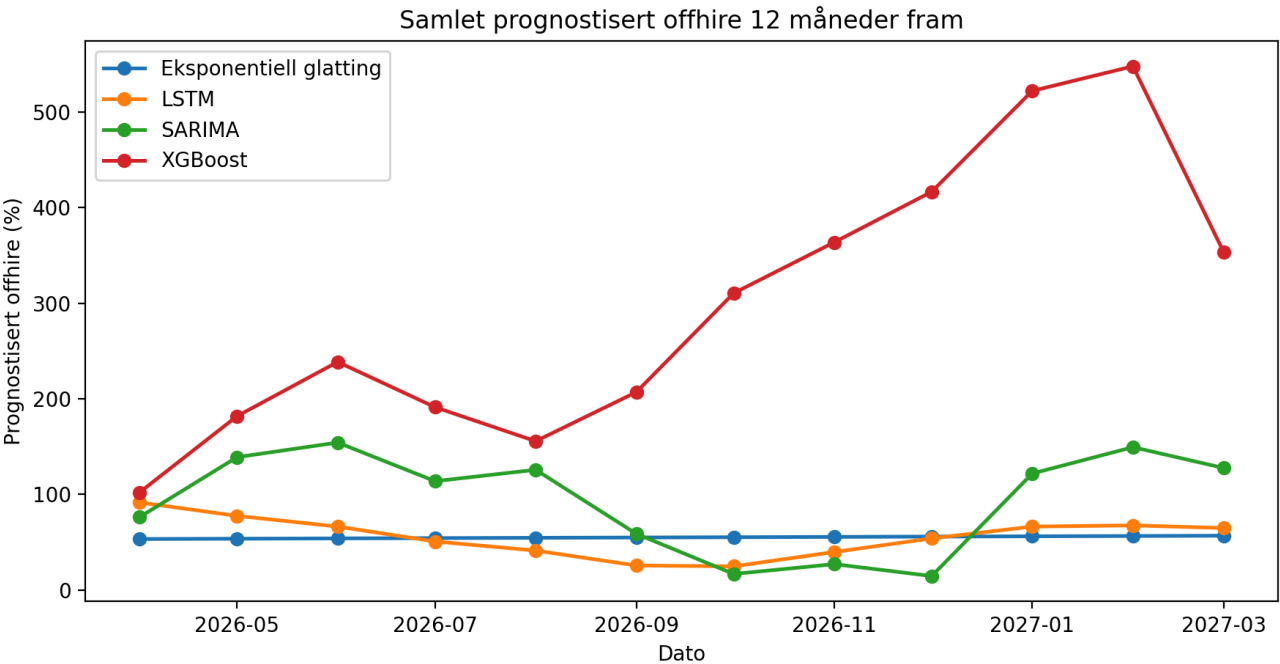
Figur 20. Samlet prognostisert offhire fra april til september 2026 for de fire modellene.

7.2.4 Prognose 12 måneder fram

Tabell 13 viser det fulle tolv månedersvinduet fram til mars 2027. Her blir modellforskjellene svært tydelige. Eksponentiell glatting holder seg nesten helt flatt mellom 53.37 og 56.84, mens LSTM først faller og deretter stiger moderat igjen mot slutten av perioden. ARIMA/SARIMA beholder et mer bølgende og sesongpreget forløp med tydelige topper i mai-juni 2026 og januar-februar 2027. XGBoost skiller seg klart ut som den mest aggressive modellen, med en topp på 547.73 i februar 2027.

Tabell 13. Samlet prognostisert offhire 12 måneder fram	Eksponentiell glatting	LSTM	ARIMA/SARIMA	XGBoost
2026-04	53.37	91.60	76.35	102.19
2026-05	53.69	77.58	138.96	181.85
2026-06	54.00	66.39	154.27	238.57
2026-07	54.32	50.80	113.85	191.26
2026-08	54.63	41.29	125.77	155.66
2026-09	54.95	25.67	58.83	206.98
2026-10	55.26	24.81	16.84	310.44
2026-11	55.58	39.80	27.08	363.72
2026-12	55.89	54.01	14.65	416.52
2027-01	56.21	66.39	121.88	522.01

Tabell 13. Samlet prognostisert offhire 12 måneder fram	Ekspontentiell glatting	LSTM	ARIMA/SARIMA	XGBoost
2027-02	56.52	67.61	149.42	547.73
2027-03	56.84	64.86	127.60	353.12



Figur 21. Samlet prognostisert offhire fra april 2026 til mars 2027 for de fire modellene.

Samlet viser fremtidsprognosene at modellene gir ganske ulike framtidsbilder, spesielt når horisonten blir lengre. På kort sikt peker de alle mot at offhire fortsatt vil være konsentrert rundt noen få fartøy, men på lengre sikt varierer både nivå og utviklingsform betydelig. Dette betyr at de historiske testresultatene blir viktige som tolkningsramme: prognosene bør ikke leses isolert, men i lys av hvilken modell som faktisk presterte best på kjent historikk.

8.0 Diskusjon

Diskusjonsdelen tolker her funnene opp mot problemstillingen, tidligere forskning og studiens metodiske forutsetninger. Formålet er ikke å gjenta resultatene, men å vurdere hva de faktisk betyr for modellvalg, praktisk beslutningsstøtte og hvor langt funnene kan strekkes utover den konkrete casen.

8.1 Modellvalg og prediksjonsnøyaktighet

Det tydeligste hovedfunnet i studien er at valg av prognosemodell faktisk påvirker prediksjonsnøyaktigheten, men ikke på en måte som gir automatisk fordel til de mest komplekse modellene. ARIMA/SARIMA oppnådde lavest MAE og lavest RMSE i den historiske testen, mens XGBoost og LSTM presterte konkurransedyktig uten å overgå

den beste klassiske modellen. Eksponentiell glatting fungerte som en stabil og transparent benchmark, men kom svakere ut enn de tre øvrige modellene. Samlet tyder dette på at datastrukturen i denne studien fortsatt belønner modeller som kan utnytte fartøyspesifikk tidsseriedynamikk på en presis og parsimonisk måte.

Dette er i tråd med Schmid et al. (2025), som viser at modellprestasjon i stor grad avhenger av problemstruktur og ikke bare av modelltype. Funnene støtter også Kolassa (2022), som argumenterer for at høy modellkompleksitet ikke automatisk gir størst praktisk verdi. Samtidig viser resultatene at maskinlærings- og dyp læringsmodeller ikke bør avskrives. At **XGBoost** og **LSTM** ligger relativt nær **ARIMA/SARIMA**, viser at de faktisk fanger vesentlige deler av mønsteret i datasettet. Studien gir derfor ikke grunnlag for å hevde at klassiske modeller alltid er best, men den viser at de i denne konkrete casen framstår som det mest forsvarlige førstevalget.

8.2 Datastruktur, marked og hvorfor resultatene ble som de ble

En sentral forklaring på resultatene ligger i selve datamaterialet. Den deskriptive analysen viste at datasettet er nulltungt, høyreskjevt og preget av store forskjeller mellom fartøyene. I tillegg opptrer offhire ofte som episodiske topper snarere enn som jevne mønstre over tid. Dette betyr at modellene ikke bare konkurrerer om å lære et nivå eller en trend, men om å håndtere et problem der lange perioder med nullverdier avbrytes av enkelte kraftige utslag. Når forskjellene mellom fartøyene samtidig er store, blir modellprestasjon også et spørsmål om hvor godt hver modell håndterer heterogenitet, ikke bare tidsavhengighet.

Dette bidrar til å forklare hvorfor **ARIMA/SARIMA** kom best ut historisk. Ved å estimeres per fartøy og verifiseres med residualdiagnostikk kan modellen i større grad tilpasses hver enkelt series struktur. Samtidig kan dette også forklare hvorfor **XGBoost** og **LSTM** ikke fikk en tydelig fordel, til tross for større fleksibilitet. Det relativt korte datagrunnlaget og den store andelen nullmåneder kan ha begrenset hvor mye ekstra kompleksitet disse modellene faktisk kunne utnytte på en robust måte.

Resultatene må også forstås i lys av at offshoresegmentet opererer i et volatilt marked. Rederier i olje- og gassrelatert aktivitet påvirkes indirekte av svingninger i energipriser, investeringsnivå, kontraktsaktivitet og globale forhold (Menon Economics, 2026). Studien modellerer ikke slike drivere eksplisitt, men de er en viktig del av bakgrunnen for at operasjonell tilgjengelighet ikke nødvendigvis utvikler seg jevnt over tid. Det betyr at datasettets uregelmessighet ikke bare er et teknisk dataproblem, men også et uttrykk for at fartøyene opererer i en usikker og skiftende kontekst.

8.3 Fremtidsprognoser og praktisk tolkning

Fremtidsprognosene må tolkes annerledes enn den historiske testen. I testperioden finnes en kjent fasit, og modellene kan rangeres etter faktisk prediksjonsfeil. For prognoseperioden finnes ingen observasjoner ennå, og resultatene blir derfor scenariobeskrivelser snarere enn verifiserte utfall. Det viktigste poenget er at modellene spriker betydelig mer jo lengre prognosehorisonten blir. Dette gjelder særlig **XGBoost**, som genererer et markant høyere langtidsforløp enn de andre modellene, mens eksponentiell glatting forblir nærmest flat gjennom hele perioden.

Dette spriket betyr ikke nødvendigvis at én modell er feil og de andre riktige. Det viser først og fremst at usikkerheten øker når prognosehorisonten forlenges. I en næring preget av stor markedsmessig volatilitet blir dette særlig viktig. Ramebetingelsene kan endre seg raskt, og studien inkluderer ikke eksterne drivere som kan fange opp slike skift direkte. Derfor framstår de kortere prognosehorisontene på 1 og 3 måneder som mer praktisk anvendelige enn 12-månedersprognosene. For Simon Møkster Shipping AS betyr dette at prognosene først og fremst bør brukes som beslutningsstøtte for kortsiktig kapasitetsplanlegging, oppfølging av fartøy med høy historisk offhire og som et supplement til operasjonell vurdering, ikke som et automatisk beslutningsgrunnlag.

8.4 Metodiske styrker og svakheter

Studien har flere metodiske styrker. For det første sammenlignes alle modellene på samme historiske tidsvindu og med samme ekspanderende 1-steps evalueringslogikk. Dette styrker den interne sammenlignbarheten og gjør at forskjeller i ytelse i større grad kan tilskrives modellene selv. For det andre kombinerer studien to klassiske og to KI-baserte modeller, noe som gir et bredere og mer faglig interessant sammenligningsgrunnlag enn om bare én modellfamilie var vurdert. For det tredje er den historiske testen holdt atskilt fra fremtidsprognosene, noe som tydeliggjør skillet mellom verifiserbar modellprestasjon og ikke-verifiserte framtidsestimater.

Samtidig er svakheterne reelle. Dataserien er relativt kort, og materialet er preget av mange nullperioder og enkelte ekstreme topper. Dette gjør både modelltrening og evaluering mer krevende. Datagrunnlaget består også av sekundærdata som ikke kan verifiseres fullt ut eksternt. I tillegg er studien avgrenset til ett rederi og ett offshoresegment, noe som begrenser den statistiske generaliserbarheten. En annen viktig begrensning er at eksterne drivere som energipriser, kontraktmarked og geopolitisk uro ikke er eksplisitt modellert. De kan bare fanges indirekte gjennom historiske observasjoner. Dette er særlig relevant for langtidsprognosene, der usikkerheten naturlig blir større. Studien er derfor best forstått som en prediktiv, casebasert sammenligning og ikke som en kausal analyse av hva som skaper offhire.

8.5 Betydning for bedriften og samlet vurdering

For Simon Møkster Shipping AS har funnene først og fremst verdi fordi de viser at modellvalg bør være et eksplisitt beslutningsspørsmål og ikke bare et teknisk implementeringsvalg. Resultatene tyder på at en klassisk tidsseriemodell, særlig **ARIMA/SARIMA**, per nå er det mest forsvarlige hovedverktøyet for historisk prediksjon av offhire i denne casen. Samtidig viser de konkurransedyktige resultatene til **XGBoost** og **LSTM** at det er faglig relevant å videreutvikle slike modeller dersom datagrunnlaget blir rikere eller mer omfattende over tid.

Opp mot problemstillingen gir studien et tydelig svar. Valg av prognosemodell påvirker prediksjonsnøyaktigheten for offhire-hendelser, men effekten av modellvalget må forstås i lys av datastruktur og kontekst. I dette datasettet presterer klassiske tidsseriemodeller best, mens KI-baserte modeller ikke gir en tydelig merverdi i historisk test. Samtidig viser fremtidsprognosene at modellene produserer ulike framtidsbilder, noe som understreker behovet for å bruke prognoser med faglig skjønn i et marked preget av betydelig usikkerhet.

9.0 Konklusjon

I denne oppgaven ble det undersøkt hvordan valg av prognosemodell påvirker prediksjonsnøyaktigheten for offhire-hendelser for fartøy innenfor samme offshoresegment. Med utgangspunkt i historiske data fra Simon Møkster Shipping AS ble to klassiske tidsseriemodeller, **ARIMA/SARIMA** og **eksponentiell glatting**, sammenlignet med to KI-baserte modeller, **XGBoost** og **LSTM**.

Hovedfunnene viser at modellvalg faktisk påvirker prediksjonsnøyaktigheten, men ikke på en måte som gir automatisk fordel til de mest komplekse modellene. I denne studien presterte **ARIMA/SARIMA** best i den historiske testen, målt ved både **MAE** og **RMSE**. **XGBoost** og **LSTM** var konkurransedyktige, men ga ikke tydelig bedre resultater enn den beste klassiske modellen, mens **eksponentiell glatting** fungerte som en nyttig, men svakere benchmark. Problemstillingen kan dermed besvares med at valg av modell har betydning, og at klassiske tidsseriemodeller i dette datasettet ga høyest prediksjonsnøyaktighet.

For casebedriften betyr dette at **ARIMA/SARIMA** per nå framstår som det mest forsvarlige hovedverktøyet for historisk prediksjon av offhire. Samtidig viser fremtidsprognosene at modellene gir ulike framtidsbilder, særlig på lengre horisonter. Prognoser bør derfor brukes som beslutningsstøtte og ikke som et automatisk

beslutningsgrunnlag, spesielt i et marked preget av betydelig volatilitet og usikre rammebetingelser.

Videre forskning bør undersøke om resultatene endrer seg når modellene testes på lengre tidsserier, rikere datagrunnlag og flere forklaringsvariabler, som kontraktsdata, tekniske indikatorer eller markedsforhold. Det vil også være relevant å evaluere fremtidsprognosene når nye observasjoner foreligger, for å se om de samme modellforskjellene består over tid.

10.0 Bibliografi

Carbonneau, R., Laframboise, K., & Vahidov, R. (2008). Application of machine learning techniques for supply chain demand forecasting. *European Journal of Operational Research*, 184(3), 1140-1154.

<https://doi.org/10.1016/j.ejor.2006.12.004>

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>

Chu, Z., Yan, R., & Wang, S. (2024). Vessel turnaround time prediction: A machine learning approach. *Ocean & Coastal Management*, 249, 107021. <https://doi.org/10.1016/j.ocecoaman.2024.107021>

Douaioui, K., Oucheikh, R., Benmoussa, O., & Mabrouki, C. (2024). Machine learning and deep learning models for demand forecasting in supply chain management: A critical review. *Applied System Innovation*, 7(5), 93. <https://doi.org/10.3390/asi7050093>

Fildes, R., Kolassa, S., & Ma, S. (2022). Post-script: Retail forecasting: Research and practice. *International Journal of Forecasting*, 38(4), 1319-1324. <https://doi.org/10.1016/j.ijforecast.2021.09.012>

Gardner, E. S., Jr. (1985). Exponential smoothing: The state of the art. *Journal of Forecasting*, 4(1), 1-28. <https://doi.org/10.1002/for.3980040103>

Hochreiter, S., & Schmidhuber, J. (1995). *Long short-term memory* (Technical Report FKI-207-95). Technische Universität München.

Hyndman, R. J., & Khandakar, Y. (2008). Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3), 1-22. <https://doi.org/10.18637/jss.v027.i03>

Hyndman, R. J., Koehler, A. B., Snyder, R. D., & Grose, S. (2002). A state space framework for automatic forecasting using exponential smoothing methods. *International Journal of Forecasting*, 18(3), 439-454. [https://doi.org/10.1016/S0169-2070\(01\)00110-8](https://doi.org/10.1016/S0169-2070(01)00110-8)

Kalafatelis, A. S., Nomikos, N., Giannopoulos, A., Alexandridis, G., Karditsa, A., & Trakadas, P. (2025). Towards predictive maintenance in the maritime industry: A component-based overview. *Journal of Marine Science and Engineering*, 13(3), 425. <https://doi.org/10.3390/jmse13030425>

Kjeldsberg, F., & Munim, Z. H. (2024). Automated machine learning driven model for predicting platform supply vessel freight market. *Computers & Industrial Engineering*, 191, 110153. <https://doi.org/10.1016/j.cie.2024.110153>

Kolassa, S. (2022). Commentary on the M5 forecasting competition. *International Journal of Forecasting*, 38(4), 1562-1568. <https://doi.org/10.1016/j.ijforecast.2021.08.006>

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4), 1346-1364.
<https://doi.org/10.1016/j.ijforecast.2021.11.013>

Menon Economics. (2026). *Maritim verdiskapingsrapport 2026* [Report].

Schmid, L., Roidl, M., Kirchheim, A., & Pauly, M. (2025). Comparing statistical and machine learning methods for time series forecasting in data-driven logistics: A simulation study. *Entropy*, 27(1), 25.
<https://doi.org/10.3390/e27010025>

11.0 Vedlegg

11.1 Oversikt over figurer

Tabell 14 gir en samlet oversikt over figurene som er brukt i rapporten, hva de viser og hvor de er omtalt.

Figur	Tittel	Kort beskrivelse	Plassering i rapporten
Figur 1	Samlet offhire per måned aggregert på tvers av alle fartøy	Viser samlet historisk offhire på tvers av fartøy	4.0 Casebeskrivelse
Figur 2	Heatmap for offhire per fartøy og måned	Viser variasjon mellom fartøy og over tid	4.0 Casebeskrivelse
Figur 3	Gjennomsnittlig månedlig offhire per fartøy	Rangerer fartøy etter gjennomsnittlig nivå	5.2.2 Deskriptiv analyse av datasettet
Figur 4	Boksplott for offhire per fartøy	Viser fordeling, median og ekstreme utslag	5.2.2 Deskriptiv analyse av datasettet
Figur 5	Tidsserier for fartøy med høyest gjennomsnittlig offhire	Viser utviklingen for de mest utsatte fartøyene	5.2.2 Deskriptiv analyse av datasettet
Figur 6	ACF for representativ ARIMA/SARIMA-serie	Støtter identifikasjon av representativ SARIMA-modell	6.1 SARIMA
Figur 7	PACF for representativ ARIMA/SARIMA-serie	Støtter identifikasjon av representativ SARIMA-modell	6.1 SARIMA
Figur 8	Residualdiagnostikk for representativ ARIMA/SARIMA-modell	Viser residualforløp og residualfordeling	6.1 SARIMA
Figur 9	Representativ testprognose for ARIMA/SARIMA	Sammenligner testprediksjon med faktisk forløp	6.1 SARIMA
Figur 10	Representativ testprognose for eksponentiell glatting	Viser modellens testforløp for representativt fartøy	6.2 Eksponentiell glatting

Figur	Tittel	Kort beskrivelse	Plassering i rapporten
Figur 11	XGBoost feature importance	Viser hvilke features som betyr mest i modellen	6.3 XGBoost
Figur 12	Representativ testprognose for XGBoost	Viser modellens testprediksjoner for representativt fartøy	6.3 XGBoost
Figur 13	Treningshistorikk for LSTM	Viser trenings- og valideringstap over epoker	6.4 LSTM
Figur 14	Representativ testprognose for LSTM	Viser modellens testprediksjoner for representativt fartøy	6.4 LSTM
Figur 15	MAE per modell i testperioden	Samlet sammenligning av modellene på MAE	7.1 Resultater fra historisk modelltesting
Figur 16	MAE per testmåned og modell	Viser hvordan prediksjonsfeilen varierer over tid	7.1 Resultater fra historisk modelltesting
Figur 17	Heatmap for MAE per fartøy og modell	Viser feilfordeling mellom fartøy og modeller	7.1 Resultater fra historisk modelltesting
Figur 18	Samlet prognostisert offhire 1 måned fram	Viser én-månedsprognosen på modellnivå	7.2 Resultater fra fremtidsprognoser
Figur 19	Samlet prognostisert offhire 3 måneder fram	Viser tre-månedersprognosen på modellnivå	7.2 Resultater fra fremtidsprognoser
Figur 20	Samlet prognostisert offhire 6 måneder fram	Viser seks-månedersprognosen på modellnivå	7.2 Resultater fra fremtidsprognoser
Figur 21	Samlet prognostisert offhire 12 måneder fram	Viser tolv-månedersprognosen på modellnivå	7.2 Resultater fra fremtidsprognoser

11.2 Oversikt over tabeller

Tabell 15 gir en samlet oversikt over tabellene som er brukt i rapporten, hva de viser og hvor de er omtalt.

Tabell	Tittel	Kort beskrivelse	Plassering i rapporten
Tabell 1	Datadekning	Oppsummerer tidsperiode, antall observasjoner og datadekning	5.2.1 Datagrunnlag
Tabell 2	Fartøy med høyest gjennomsnittlig offhire	Viser topp fem fartøy etter gjennomsnittlig offhire	5.2.2 Deskriptiv analyse av datasettet

Tabell	Tittel	Kort beskrivelse	Plassering i rapporten
Tabell 3	Felles evalueringsoppsett	Oppsummerer felles testdesign for modellene	6.0 Modellering
Tabell 4	Beste kandidatmodeller for representativt fartøy (Fartøy 2)	Viser SARIMA-kandidater rangert etter AIC og BIC	6.1 SARIMA
Tabell 5	Valgt ETS-spesifikasjon i siste kjøring	Viser fordeling av valgte ETS-varianter	6.2 Eksponentiell glatting
Tabell 6	XGBoost-featuregrupper	Oppsummerer feature-settet brukt i modellen	6.3 XGBoost
Tabell 7	XGBoost-hyperparametre	Oppsummerer sentrale hyperparametre	6.3 XGBoost
Tabell 8	LSTM-oppsett i siste kjøring	Oppsummerer sekvenslengde, inputfeatures og arkitektur	6.4 LSTM
Tabell 9	Samlet testresultat for modellene	Viser MAE, RMSE og SMAPE for alle modeller	7.1 Resultater fra historisk modelltesting
Tabell 10	Samlet prognostisert offhire 1 måned fram	Viser én-månedsprognozen for alle modeller	7.2 Resultater fra fremtidsprognoser
Tabell 11	Samlet prognostisert offhire 3 måneder fram	Viser tre-månedersprognosen for alle modeller	7.2 Resultater fra fremtidsprognoser
Tabell 12	Samlet prognostisert offhire 6 måneder fram	Viser seks-månedersprognosen for alle modeller	7.2 Resultater fra fremtidsprognoser
Tabell 13	Samlet prognostisert offhire 12 måneder fram	Viser tolv-månedersprognosen for alle modeller	7.2 Resultater fra fremtidsprognoser

11.3 Kodevedlegg

Kodevedleggene nedenfor viser modellspesifikke funksjonsuttrekk fra den aktive implementasjonen i modelleringsmappen under `004 data/modeling/`. Hensikten er å dokumentere hvordan hver modell er implementert, uten å gjengi hele kodebasen i vedlegget.

11.3.1 SARIMA-kode

Vedlegg 11.3.1 viser funksjonen `run_sarima`, som står for fartøyvis modellvalg, residualdiagnostikk og ekspanderende 1-steps prediksjon for ARIMA/SARIMA.

```
def run_sarima(
    train_panel: pd.DataFrame,
    test_panel: pd.DataFrame,
    split_metadata: dict[str, Any],
```

```

) -> tuple[ModelResult, pd.DataFrame]:
    prediction_rows: list[dict[str, Any]] = []
    stationarity_rows: list[dict[str, Any]] = []
    model_rows: list[dict[str, Any]] = []
    residual_rows: list[dict[str, Any]] = []
    representative_vessel = select_representative_vessel(train_panel)
    representative_candidates = pd.DataFrame()
    representative_transformed: pd.Series | None = None
    representative_residuals: pd.Series | None = None
    fallback_representative: str | None = None

    train_vessels =
        sorted(set(train_panel["vessel"]).intersection(test_panel["vessel"]))
    for vessel in train_vessels:
        train_series = build_vessel_series(train_panel, vessel).dropna()
        test_series = build_vessel_series(test_panel, vessel).dropna()
        if len(train_series) < 24 or test_series.empty:
            continue

        if train_series.nunique() <= 1:
            constant_value = float(train_series.iloc[-1])
            model_rows.append(
                {
                    "vessel": vessel,
                    "p": np.nan,
                    "d": np.nan,
                    "q": np.nan,
                    "P": np.nan,
                    "D": np.nan,
                    "Q": np.nan,
                    "s": 0,
                    "aic": np.nan,
                    "bic": np.nan,
                    "train_observations": int(len(train_series)),
                }
            )
            residual_rows.append(
                {
                    "vessel": vessel,
                    "ljung_box_lag": np.nan,
                    "ljung_box_pvalue": np.nan,
                }
            )
        for date_value, actual in test_series.items():
            prediction_rows.append(
                {
                    "model": "sarima",
                    "vessel": vessel,

```

```

        "date": pd.Timestamp(date_value).strftime("%Y-%m-%d"),
        "actual": float(actual),
        "prediction": constant_value,
    }
)
    continue

```

```

selected_d, selected_D, transformed_series, stationarity_results =
select_sarima_differencing(
    train_series
)
stationarity_rows.extend(
    {"vessel": vessel, **row} for row in stationarity_results
)
candidate_df = fit_sarima_candidates(train_series, d=selected_d,
D=selected_D)
best_candidate = candidate_df.iloc[0]
selected_order = (
    int(best_candidate["p"]),
    int(best_candidate["d"]),
    int(best_candidate["q"]),
)
selected_seasonal_order = (
    int(best_candidate["P"]),
    int(best_candidate["D"]),
    int(best_candidate["Q"]),
    int(best_candidate["s"]),
)

final_model = SARIMAX(
    train_series,
    order=selected_order,
    seasonal_order=selected_seasonal_order,
    enforce_stationarity=False,
    enforce_invertibility=False,
)
final_fit = final_model.fit(dispatch=False)
residuals = pd.Series(final_fit.resid,
index=train_series.index).dropna()
ljung_lag = min(12, max(len(residuals) // 2, 1))
ljung_box = acorr_ljungbox(residuals, lags=[ljung_lag],
return_df=True)

model_rows.append(
    {
        "vessel": vessel,
        "p": selected_order[0],
        "d": selected_order[1],
        "q": selected_order[2],
    }
)

```

```

        "P": selected_seasonal_order[0],
        "D": selected_seasonal_order[1],
        "Q": selected_seasonal_order[2],
        "s": selected_seasonal_order[3],
        "aic": float(best_candidate["aic"]),
        "bic": float(best_candidate["bic"]),
        "train_observations": int(len(train_series)),
    }
)
residual_rows.append(
    {
        "vessel": vessel,
        "ljung_box_lag": int(ljung_lag),
        "ljung_box_pvalue": float(ljung_box["lb_pvalue"].iloc[0]),
    }
)

history = train_series.copy()
for date_value, actual in test_series.items():
    prediction = float(
        fit_or_fallback_sarima_forecast(
            history,
            1,
            order=selected_order,
            seasonal_order=selected_seasonal_order,
        )[0]
    )
    prediction_rows.append(
        {
            "model": "sarima",
            "vessel": vessel,
            "date": pd.Timestamp(date_value).strftime("%Y-%m-%d"),
            "actual": float(actual),
            "prediction": prediction,
        }
    )
    history = pd.concat(
        [history, pd.Series([float(actual)],
index=pd.DatetimeIndex([date_value]))]
    )

if fallback_representative is None:
    fallback_representative = vessel
if representative_vessel == vessel or (
    representative_vessel not in train_vessels and
    fallback_representative == vessel
):
    representative_candidates = candidate_df.head(10).copy()
    representative_transformed = transformed_series

```

```
representative_residuals = residuals
representative_vessel = vessel

if representative_candidates.empty and fallback_representative is not
    None:
    representative_vessel = fallback_representative

if not prediction_rows:
    return (
        ModelResult(
            model="sarima",
            status="skipped",
            details={
                **split_metadata,
                "reason": "Ingen fartøy hadde tilstrekkelig historikk og
variasjon til ARIMA/SARIMA.",
            },
        ),
        pd.DataFrame(),
    )

pred_df = pd.DataFrame(prediction_rows)
write_dataframe_artifacts(
    pd.DataFrame(stationarity_rows),
    model_artifact_path("sarima", "stasjonaritet.csv"),
    "ARIMA/SARIMA stasjonaritet per fartøy",
)
write_dataframe_artifacts(
    representative_candidates,
    model_artifact_path("sarima", "kandidatmodeller.csv"),
    "ARIMA/SARIMA kandidatmodeller for representativt fartøy",
)
write_dataframe_artifacts(
    pd.DataFrame(model_rows),
    model_artifact_path("sarima", "modellvalg_per_fartoy.csv"),
    "ARIMA/SARIMA modellvalg per fartøy",
)
write_dataframe_artifacts(
    pd.DataFrame(residual_rows),
    model_artifact_path("sarima", "residualdiagnostikk.csv"),
    "ARIMA/SARIMA residualdiagnostikk per fartøy",
)
if (
    representative_vessel is not None
    and representative_transformed is not None
    and representative_residuals is not None
):
    save_sarima_diagnostic_plots(
        representative_vessel,
```

```

        representative_transformed,
        representative_residuals,
    )
    save_representative_prediction_plot(
        "sarima",
        representative_vessel,
        train_panel,
        test_panel,
        pred_df,
        "ARIMA/SARIMA: representativt testforløp",
    )

    mae_value, rmse_value, smape_value = summarize_prediction_frame(pred_df)
    metrics = ModelResult(
        model="sarima",
        status="ok",
        mae=mae_value,
        rmse=rmse_value,
        smape=smape_value,
        details={
            **split_metadata,
            "series_type": "per_vessel",
            "test_rows": int(len(pred_df)),
            "evaluation_method": "eksperderende 1-steps prognose per
fartøy",
            "evaluation_level": "fartøynivå",
            "modeled_vessels": int(pred_df["vessel"].nunique()),
            "representative_vessel": representative_vessel,
            "artifact_files": {
                "stasjonaritet": "stasjonaritet.md",
                "kandidatmodeller": "kandidatmodeller.md",
                "modellvalg_per_fartoy": "modellvalg_per_fartoy.md",
                "residualdiagnostikk_tabell": "residualdiagnostikk.md",
                "acf": "acf.png",
                "pacf": "pacf.png",
                "residualdiagnostikk_figur": "residualdiagnostikk.png",
                "representativ_testplot": "representativ_testplot.png",
            },
        },
    )
    return metrics, pred_df

```

11.3.2 Eksponentiell glatting-kode

Vedlegg 11.3.2 viser funksjonen `run_exponential_smoothing`, som står for fartøyvis modellvalg mellom ETS-varianter og eksperderende 1-steps prediksjon gjennom testperioden.

```

def run_exponential_smoothing(
    train_panel: pd.DataFrame,
    test_panel: pd.DataFrame,

```

```

split_metadata: dict[str, Any],
) -> tuple[ModelResult, pd.DataFrame]:
    predictions: list[dict[str, Any]] = []
    model_selection_rows: list[dict[str, Any]] = []
    residual_rows: list[dict[str, Any]] = []
    representative_vessel = select_representative_vessel(train_panel)

    for vessel, test_vessel_df in test_panel.groupby("vessel"):
        train_series = build_vessel_series(train_panel, vessel).dropna()
        test_series = build_vessel_series(test_panel, vessel).dropna()
        if len(train_series) < 12 or test_series.empty:
            continue

        if train_series.nunique() <= 1:
            constant_value = float(train_series.iloc[-1])
            model_selection_rows.append(
                {
                    "vessel": vessel,
                    "spec": "CONST",
                    "aic": np.nan,
                    "bic": np.nan,
                    "train_observations": int(len(train_series)),
                }
            )
            residual_rows.append(
                {
                    "vessel": vessel,
                    "ljung_box_lag": np.nan,
                    "ljung_box_pvalue": np.nan,
                }
            )
            for date_value, actual in test_series.items():
                predictions.append(
                    {
                        "model": "exponential_smoothing",
                        "vessel": vessel,
                        "date": pd.Timestamp(date_value).strftime("%Y-%m-%d"),
                        "actual": float(actual),
                        "prediction": constant_value,
                    }
                )
            continue

    candidate_df = fit_ets_candidates(train_series)
    best_candidate = candidate_df.iloc[0]
    fit = fit_ets_model(
        train_series,
        trend=normalize_optional_string(best_candidate["trend"]),

```



```

seasonal=normalize_optional_string(best_candidate["seasonal"]),
seasonal_periods=(
    int(best_candidate["seasonal_periods"])
    if pd.notna(best_candidate["seasonal_periods"])
    else None
),
)
residuals = pd.Series(fit.resid, index=train_series.index).dropna()
ljung_lag = min(12, max(len(residuals) // 2, 1))
ljung_box = acorr_ljungbox(residuals, lags=[ljung_lag],
return_df=True)

model_selection_rows.append(
    {
        "vessel": vessel,
        "spec": best_candidate["spec"],
        "aic": float(best_candidate["aic"]),
        "bic": float(best_candidate["bic"]),
        "train_observations": int(len(train_series)),
    }
)
residual_rows.append(
    {
        "vessel": vessel,
        "ljung_box_lag": int(ljung_lag),
        "ljung_box_pvalue": float(ljung_box["lb_pvalue"].iloc[0]),
    }
)

history = train_series.copy()
for date_value, actual in test_series.items():
    pred = float(
        fit_or_fallback_exponential_forecast(
            history,
            1,

trend=normalize_optional_string(best_candidate["trend"]),

seasonal=normalize_optional_string(best_candidate["seasonal"]),
    seasonal_periods=(
        int(best_candidate["seasonal_periods"])
        if pd.notna(best_candidate["seasonal_periods"])
        else None
    ),
    )[0]
)
predictions.append(
    {
        "model": "exponential_smoothing",

```

```

        "vessel": vessel,
        "date": pd.Timestamp(date_value).strftime("%Y-%m-%d"),
        "actual": float(actual),
        "prediction": pred,
    }
)
history = pd.concat(
    [history, pd.Series([float(actual)],
index=pd.DatetimeIndex([date_value]))]
)

if not predictions:
    return (
        ModelResult(
            model="exponential_smoothing",
            status="skipped",
            details={
                **split_metadata,
                "reason": "For få observasjoner per fartøy til å kjøre
eksponentiell glatting.",
            },
        ),
        pd.DataFrame(),
    )

pred_df = pd.DataFrame(predictions)
write_dataframe_artifacts(
    pd.DataFrame(model_selection_rows),
    model_artifact_path("exponential_smoothing",
        "modellvalg_per_fartoy.csv"),
    "Eksponentiell glatting modellvalg per fartøy",
)
write_dataframe_artifacts(
    pd.DataFrame(residual_rows),
    model_artifact_path("exponential_smoothing",
        "residualdiagnostikk.csv"),
    "Eksponentiell glatting residualdiagnostikk",
)
save_representative_prediction_plot(
    "exponential_smoothing",
    representative_vessel,
    train_panel,
    test_panel,
    pred_df,
    "Eksponentiell glatting: representativt testforløp",
)
mae_value, rmse_value, smape_value = summarize_prediction_frame(pred_df)
metrics = ModelResult(
    model="exponential_smoothing",

```

```

status="ok",
mae=mae_value,
rmse=rmse_value,
smape=smape_value,
details={
    **split_metadata,
    "vessels_used": int(pred_df["vessel"].nunique()),
    "test_rows": int(len(pred_df)),
    "evaluation_method": "ekspanderende 1-steps prognose gjennom
testperioden",
    "evaluation_level": "fartøynivå",
    "representative_vessel": representative_vessel,
    "selection_table": "modellvalg_per_fartoy.md",
    "residual_table": "residualdiagnostikk.md",
},
)
return metrics, pred_df

```

11.3.3 XGBoost-kode

Vedlegg 11.3.3 viser funksjonen `run_xgboost`, som står for feature-basert panelmodellering og ekspanderende 1-steps prediksjon med månedlig re-trening.

```

def run_xgboost(
    panel_df: pd.DataFrame,
    split_metadata: dict[str, Any],
) -> tuple[ModelResult, pd.DataFrame]:
    history_panel = panel_df[panel_df["date"] <= TRAIN_END_DATE].copy()
    test_panel = panel_df[panel_df["date"] >= TEST_START_DATE].copy()
    reference_train_df = build_panel_features(history_panel)
    if reference_train_df.empty:
        raise DataTooShortError("For få historiske observasjoner til å bygge
XGBoost-features.")

    reference_pipeline, feature_columns = build_xgboost_pipeline()
    reference_pipeline.fit(reference_train_df[feature_columns],
        reference_train_df["offhire_days"])
    save_feature_importance_artifacts(reference_pipeline)

    representative_vessel = select_representative_vessel(history_panel)
    predictions: list[dict[str, Any]] = []
    test_dates = sorted(test_panel["date"].unique())

    for date_value in test_dates:
        train_df = build_panel_features(history_panel)
        if train_df.empty:
            continue
        target_month_df = test_panel[test_panel["date"] ==
date_value].copy()

```

```

feature_rows = build_xgboost_feature_rows(history_panel,
target_month_df)
if feature_rows.empty:
    history_panel = pd.concat([history_panel, target_month_df],
ignore_index=True)
    history_panel = history_panel.sort_values(["vessel",
"date"]).reset_index(drop=True)
    continue

pipeline, feature_columns = build_xgboost_pipeline()
pipeline.fit(train_df[feature_columns], train_df["offhire_days"])
step_predictions = np.clip(
    pipeline.predict(feature_rows[feature_columns]),
    0.0,
    MAX_OFFHIRE_VALUE,
)
for _, row, prediction in zip(feature_rows.index,
feature_rows.itertuples(index=False), step_predictions):
    predictions.append(
        {
            "model": "xgboost",
            "vessel": row.vessel,
            "date": pd.Timestamp(row.date).strftime("%Y-%m-%d"),
            "actual": float(row.actual),
            "prediction": float(prediction),
        }
    )

history_panel = pd.concat([history_panel, target_month_df],
ignore_index=True)
history_panel = history_panel.sort_values(["vessel",
"date"]).reset_index(drop=True)

pred_df = pd.DataFrame(predictions)
save_representative_prediction_plot(
    "xgboost",
    representative_vessel,
    panel_df[panel_df["date"] <= TRAIN_END_DATE],
    test_panel,
    pred_df,
    "XGBoost: representativt testforløp",
)
mae_value, rmse_value, smape_value = summarize_prediction_frame(pred_df)
metrics = ModelResult(
    model="xgboost",
    status="ok",
    mae=mae_value,
    rmse=rmse_value,
    smape=smape_value,
    details={

```

```

        **split_metadata,
        "train_rows": int(len(reference_train_df)),
        "test_rows": int(len(pred_df)),
        "walk_forward_steps": int(len(test_dates)),
        "evaluation_method": "ekspanderende 1-steps prognose med
månedlig re-trening",
        "evaluation_level": "fartøynivå",
        "feature_columns": feature_columns,
        "representative_vessel": representative_vessel,
        "model_hyperparameters": {
            "n_estimators": 200,
            "max_depth": 4,
            "learning_rate": 0.05,
            "subsample": 0.9,
            "colsample_bytree": 0.9,
        },
    },
)
return metrics, pred_df

```

11.3.4 LSTM-kode

Vedlegg 11.3.4 viser funksjonen `run_lstm`, som står for sekvensbygging, månedlig re-trening og ekspanderende 1-steps prediksjon for den globale LSTM-modellen.

```

def run_lstm(
    panel_df: pd.DataFrame,
    split_metadata: dict[str, Any],
) -> tuple[ModelResult, pd.DataFrame]:
    history_panel = panel_df[panel_df["date"] <= TRAIN_END_DATE].copy()
    test_panel = panel_df[panel_df["date"] >= TEST_START_DATE].copy()
    representative_vessel = select_representative_vessel(history_panel)
    window_size = 12

    X_reference, y_reference, _ = build_lstm_sequences(history_panel,
        window_size=window_size)
    _, _, _ = train_lstm_regressor(X_reference, y_reference)
    save_lstm_training_history(history_df)

    pred_records: list[dict[str, Any]] = []
    test_dates = sorted(test_panel["date"].unique())

    for date_value in test_dates:
        X_train, y_train, _ = build_lstm_sequences(history_panel,
            window_size=window_size)
        if len(X_train) == 0:
            continue

        model, x_scaler, y_scaler, _ = train_lstm_regressor(X_train,
            y_train)

```

```

month_df = test_panel[test_panel["date"] == date_value].copy()

for _, row in month_df.iterrows():
    vessel_history = (
        history_panel[history_panel["vessel"] == row["vessel"]]
        .sort_values("date")
        .reset_index(drop=True)
    )
    if len(vessel_history) < window_size:
        continue

    history_values =
vessel_history["offhire_days"].to_numpy(dtype=np.float32)[-
window_size:]
    history_months =
vessel_history["month_num"].to_numpy(dtype=np.float32)[-
window_size:]
    history_special = (
        vessel_history["Spesielle behov/krav"]
        .fillna("")
        .str.strip()
        .ne("")
        .astype(np.float32)
        .to_numpy()[-window_size:]
    )
    sequence = np.stack(
        [
            history_values,
            np.sin(2 * np.pi * history_months / 12.0),
            np.cos(2 * np.pi * history_months / 12.0),
            history_special,
        ],
        axis=1,
    )
    sequence_scaled = x_scaler.transform(sequence).reshape(1,
sequence.shape[0], sequence.shape[1])
    prediction_scaled = model.predict(sequence_scaled,
verbose=0).reshape(-1, 1)
    prediction = float(
        np.clip(

y_scaler.inverse_transform(prediction_scaled).reshape(-1)[0],
        0.0,
        MAX_OFFHIRE_VALUE,
    )
)
    pred_records.append(
        {
            "model": "lstm",
            "vessel": row["vessel"],

```

```

        "date": pd.Timestamp(row["date"]).strftime("%Y-%m-%d"),
        "actual": float(row["offhire_days"]),
        "prediction": prediction,
    }
)

history_panel = pd.concat([history_panel, month_df],
    ignore_index=True)
history_panel = history_panel.sort_values(["vessel",
    "date"]).reset_index(drop=True)

pred_df = pd.DataFrame(pred_records)
save_representative_prediction_plot(
    "lstm",
    representative_vessel,
    panel_df[panel_df["date"] <= TRAIN_END_DATE],
    test_panel,
    pred_df,
    "LSTM: representativt testforløp",
)
mae_value, rmse_value, smape_value = summarize_prediction_frame(pred_df)
metrics = ModelResult(
    model="lstm",
    status="ok",
    mae=mae_value,
    rmse=rmse_value,
    smape=smape_value,
    details={
        **split_metadata,
        "train_sequences": int(len(X_reference)),
        "test_sequences": int(len(pred_df)),
        "evaluation_method": "ekspanderende 1-steps prognose med
månedlig re-trening",
        "evaluation_level": "fartøynivå",
        "walk_forward_steps": int(len(test_dates)),
        "sequence_length": window_size,
        "input_features": ["offhire_days", "month_sin", "month_cos",
"special_flag"],
        "representative_vessel": representative_vessel,
        "architecture": {
            "lstm_units": 32,
            "dense_units": 16,
            "batch_size": 8,
            "max_epochs": 100,
        },
    },
)
return metrics, pred_df

```